



**Politecnico
di Torino**

POLITECNICO DI TORINO

Master Degree course in Machine learning for networking

Group 22

Student Name: Ameglio Alberto

Roll Number: s330946

Student Name: Di Bella Giovanni Luca

Roll Number: s332088

Student Name: Di Stasio Enrico

Roll Number: s323075

Student Name: Vergari Cosimo

Roll Number: s329479

Academic Year 2023-2024

Supervisors:

Prof. Gabriele Ciravegna

Introduction

In this project we work on the *German Credit Dataset*: a particularly well-known dataset used for credit scoring and classification problems. Our initial focus is on data analysis and preprocessing, where we encode categorical variables and normalize the data to make it suitable for machine learning tasks. We then apply both unsupervised and supervised techniques to the dataset. Our goal is to build a model capable of accurately classifying whether a person has a good or bad credit score, and then deliberately compromise this model by injecting noise into the test set to decrease its classification accuracy. We utilized both manually generated random noise and the **Adversarial Robustness Toolbox (ART)** for these attacks. In the final section, we also implemented a solution to mitigate the effects of these attacks on the model. The main tools used in this project include **Scikit-Learn, PyTorch, the Adversarial Robustness Toolbox(ART) and other minor python packages**. The project is structured in three main *.ipynb* files and two "extra" ones. The three main ones are: *Adversarial Attack.ipynb*, *Adversarial Attack Unsupervised Data Analysis.ipynb* and *Adversarial Attack Exploratory Data Analysis.ipynb*; these follow the project structure and execute all the steps in the assignment. The other two files, *Adversarial Attack Supervised Data Analysis.ipynb* and *Adversarial Attack test PCA components.ipynb* are respectively an analysis which led us to the use of **SMOTE** and a series of tests to determine the optimal number of **PCA** components.

Keywords: (security, complexity, algorithms, machine learning, information retrieval)

Contents

Introduction	i
Table of Contents	iii
1 Data Exploration and Pre-processing	1
1.1 Dataset Acquisition	1
1.1.1 Dataset	1
1.1.2 Categorical Features	1
1.1.3 Numerical Features	3
1.1.4 Label	3
1.2 Data Preprocessing	4
1.3 Exploratory Data Analysis	4
1.3.1 Dataset Imbalance	5
1.3.2 Numerical Features	5
1.3.3 Categorical Variables	11
1.3.4 Results	15
2 Unsupervised exploration and clustering	16
2.1 Dimensionality Reduction For Data Visualization	16
2.2 PCA	16
2.2.1 Theoretical foundation	17
2.2.2 Implementation	18
2.2.3 Evaluating PCA Component number through Sequential Removal and Performance Metrics	19
2.3 t-SNE	20
2.3.1 Theoretical foundation	20
2.3.2 Implementation	21
2.4 Unsupervised Data Analysis	22
2.4.1 K-Means	22
2.4.2 Gaussian Mixture Model	27

3	Supervised learning: classification	29
3.1	Preamble	29
3.1.1	SMOTE	29
3.2	Classifier Selection	31
3.3	Cross Validation	31
3.3.1	Here we consider non NN classifiers	31
3.3.2	Neural Network classifier	35
3.4	Classifier Evaluation	37
3.4.1	Non-NN Classifier	37
3.4.2	Neural Network classifier	39
4	Adversarial Attack	41
4.1	Random Noise	41
4.1.1	Performance Analysis	42
4.2	Feature Specific Noise	43
4.2.1	Performance Analysis	43
4.2.2	Most vulnerable feature	44
4.3	Adversarial Attack with ART	45
4.3.1	Fast Gradient Sign Method (FGSM) Attack	45
4.3.2	Basic Iterative Method (BIM) Attack	46
4.3.3	Projected Gradient Descent (PGD) Attack	47
4.4	Countermeasures Exploration	48
4.4.1	Adversarial Training	48
	Bibliography	50

Chapter 1

Data Exploration and Pre-processing

1.1 Dataset Acquisition

We first downloaded the "Statlog (German Credit Data)" dataset from the UC Irvine Machine Learning Repository[3]. We had at our disposal three files:

- *german.data*, containing the data in both categorical and numerical attributes
- *german.data-numeric*, containing the data in only numerical attributes
- *german.doc*, containing a guide to understand the meaning of the various attributes

We acquired the data and turned into a *Pandas DataFrame* for an easier manipulation and **pre-processing**. We will proceed introducing the features we had to deal with, describing them and evaluating their possible impact.

1.1.1 Dataset

The dataset consists of 1,000 samples, each representing a unique loan applicant described by 20 distinct features. These features include both categorical and numerical data, providing a comprehensive view of the applicants' socio-economic status. Specifically, the dataset is composed by 13 categorical features and 7 numerical features.

Categorical features are encoded using alphanumeric codes. Below are some examples of the dataset features and their meanings.

1.1.2 Categorical Features

Existing Checking Account

This feature indicates the status of the existing checking account, specifying the range of money present (if any).

Value	Description
A11	.. < 0 DM
A12	0 <= ... < 200 DM
A13	... >= 200 DM / salary assignments for at least 1 year
A14	no checking account

Note: DM stands for Deutsche Marks.

Purpose

This feature indicates the purpose of the requested loan.

Value	Description
A40	car (new)
A41	car (used)
A42	furniture/equipment
A43	radio/television
A44	domestic appliances
A45	repairs
A46	education
A47	vacation
A48	retraining
A49	business
A410	others

Personal Status and Sex

Value	Description
A91	male : divorced/separated
A92	female : divorced/separated/married
A93	male : single
A94	male : married/widowed
A95	female : single

Property

This feature indicates the type of property owned by the applicant.

Value	Description
A121	real estate
A122	if not A121 : building society savings agreement/ life insurance
A123	if not A121/A122 : car or other, not in attribute 6
A124	unknown / no property

Housing

Value	Description
A151	rent
A152	own
A153	for free

1.1.3 Numerical Features

- **Month Duration:** This feature indicates the duration in month of the asked loan.
- **Credit Amount:** This feature indicates the amount of money requested for the credit.
- **Age:** This feature indicates the age of the applicant in years.
- **Existing credit number:** This feature indicates the number of existing credit at this bank.
- **People liable:** Number of people being liable to provide maintenance for.
- **Present Residence since**
- **Install rate:** Installment rate in percentage of disposable income

1.1.4 Label

Each sample is labeled with an indication of whether the loan request was accepted or rejected by the bank, which we will use as ground truth to train and evaluate the performance of our models.

Value	Description
1	good (applicant eligible for credit)
2	bad (applicant not eligible for credit)

1.2 Data Preprocessing

We preprocessed the raw data to allow the Machine Learning models to process categorical data effectively. Since the algorithms we plan to use require numerical input, our first **pre-processing** step was to select the categorical features and encode them using **Dummy Encoding**. We opted for **Dummy Encoding** to reduce the number of categories compared to **One-Hot Encoding** because there were no missing values in the dataset.

After applying Dummy Encoding to the dataset, we ended up with 61 features. While this is generally an acceptable number of features, the limited number of records in the dataset could result in overfitting and reduce the generalization capability of our models. In the following sections, we explore common dimensionality reduction techniques to potentially reduce the number of features.

The next step was to normalize all values between 0 and 1 using the **Min-Max scaler**. Normalization was performed to enhance both the model's performance and the dataset's readability. Normalization helps to mitigate the impact of outliers and prevents numerical instability caused by large variations in feature scales.

Additionally, we modified the labels from 1 (positive) and 2 (negative) to 1 (positive) and 0 (negative) for our convenience and created a copy of the dataset in its current state for use in the Adversarial Attack section. The final step involved generating another copy of the dataset with the labels removed, which was used for exploratory data analysis and unsupervised learning.

Here is a snippet of the final dataframe:

	Duration	Credit_amount	Install_rate	Present_residence_since	Age	No_credits	No_people_liable	good/bad	Status_account_A11	Status_account_A12	...	Housing_A152	Housing_A153	Job_A171	Job_A172	Job_A173	Job_A174
0	0.029412	0.050567	1.000000	1.000000	0.857143	0.333333	0.0	1.0	1.0	0.0	...	1.0	0.0	0.0	0.0	1.0	0.0
1	0.647059	0.313690	0.333333	0.333333	0.053571	0.000000	0.0	0.0	0.0	1.0	...	1.0	0.0	0.0	0.0	1.0	0.0
2	0.117647	0.101574	0.333333	0.666667	0.535714	0.000000	1.0	1.0	0.0	0.0	...	1.0	0.0	0.0	1.0	0.0	0.0
3	0.558824	0.419941	0.333333	1.000000	0.464286	0.000000	1.0	1.0	1.0	0.0	...	0.0	1.0	0.0	0.0	1.0	0.0
4	0.294118	0.254209	0.666667	1.000000	0.607143	0.333333	1.0	0.0	1.0	0.0	...	0.0	1.0	0.0	0.0	1.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	0.117647	0.081765	0.666667	1.000000	0.214286	0.000000	0.0	1.0	0.0	0.0	...	1.0	0.0	0.0	1.0	0.0	0.0
996	0.382353	0.198470	1.000000	1.000000	0.375000	0.000000	0.0	1.0	1.0	0.0	...	1.0	0.0	0.0	0.0	0.0	1.0
997	0.117647	0.030483	1.000000	1.000000	0.339286	0.000000	0.0	1.0	0.0	0.0	...	1.0	0.0	0.0	0.0	1.0	0.0
998	0.602941	0.087763	1.000000	1.000000	0.071429	0.000000	0.0	0.0	1.0	0.0	...	0.0	1.0	0.0	0.0	1.0	0.0
999	0.602941	0.238032	0.666667	1.000000	0.142857	0.000000	0.0	1.0	0.0	1.0	...	1.0	0.0	0.0	0.0	1.0	0.0

1000 rows × 62 columns

Figure 1.1: Dataset description snippet

We also performed the train/test split reserving 20% the original dataset for testing.

1.3 Exploratory Data Analysis

We begin our analysis by examining the attributes in detail to uncover any relationships between them, using univariate and bivariate analysis. We'll utilize various graphical representations and statistical summaries to extract insights. Our goal is to identify potential correlations and gather preliminary information, providing an early empirical estimation of the dataset's most critical features that could provide valuable insights into the behavior of good and bad credit applicants.

1.3.1 Dataset Imbalance

The first element we analyze is the distribution of the two labels in the dataset. As shown in Figure 1.2 we can clearly see that there's a remarkable imbalance, with significantly more **Good** labels than **Bad** ones. This inequality can cause problems during classifier model training, and thus it needs to be addressed.

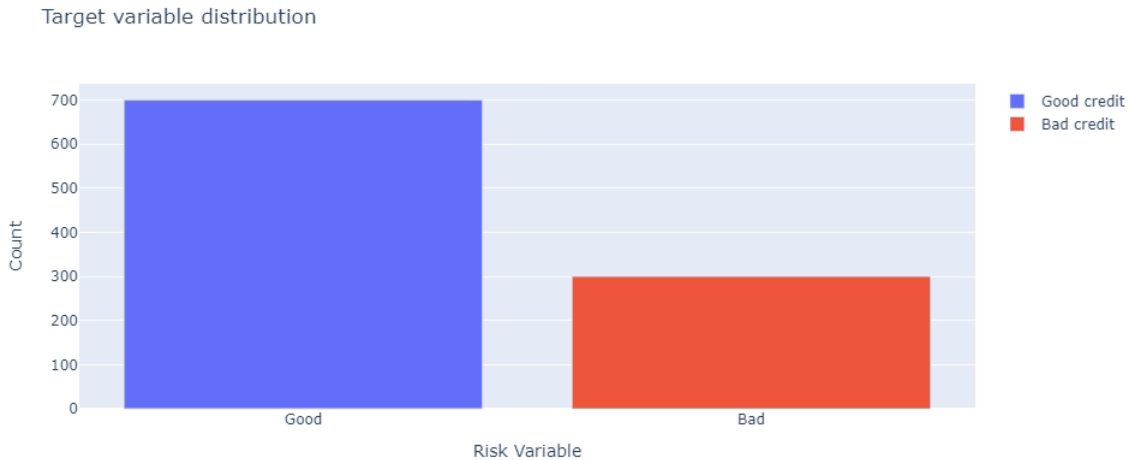


Figure 1.2: Dataset imbalance in labels

1.3.2 Numerical Features

Age Distribution Analysis

We start examining the distribution of the Age column. The interval of this features goes from 19 to 75 years. The descriptive statistics for the Age column are as follows:

```
count    1000
mean      35.54
std       11.37
min       19
25%       27
50%       33
75%       42
max       75
```

As we can see the average age of the clients in our dataset is 35 years. This age group typically falls within the prime working years for the majority of the individuals. While age is one aspect considered by banks during loan applications, it's important to remember that it is not sufficient, nor a primary factor, for a comprehensive assessment for credit risk.

As shown in Figure 1.3, to visualize the distribution, we used a histogram with 56 bins (one for each year) and an horizontal box plot to provide an interactive and comprehensive view. The median age is 33, with a noticeable peak around 28 years. Outliers are present, representing individuals older than the majority (above 70 years).

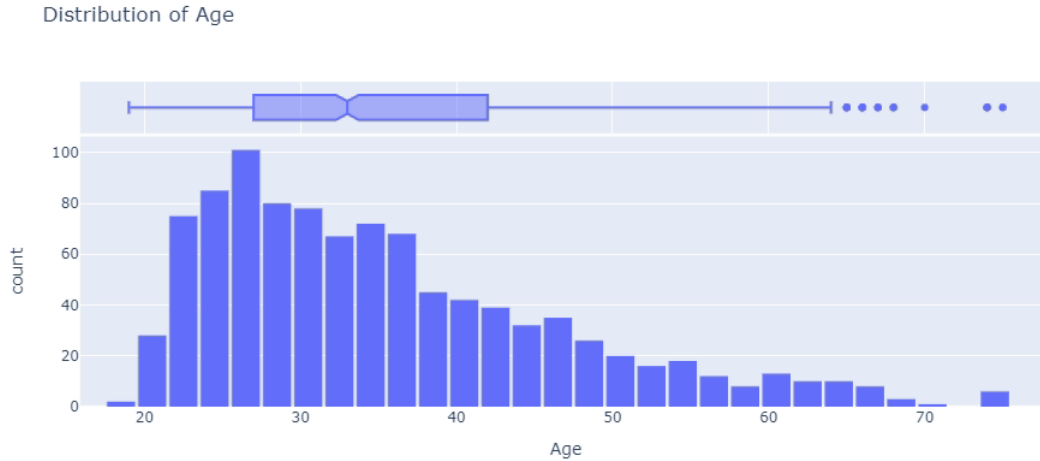


Figure 1.3: Age Distribution

From Figure 1.4 we can see that the age distribution is right-skewed, indicating a higher concentration of younger individuals over the elders. We can also note, beyond the disparity in number, that there is no significant difference in the age distribution between good creditors and bad ones.

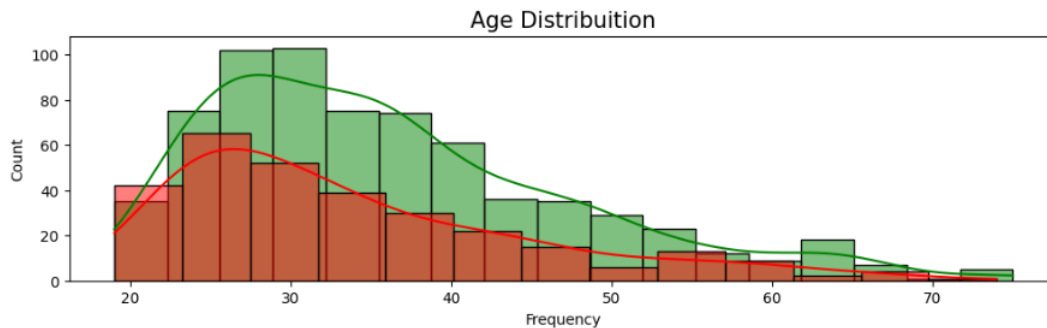


Figure 1.4: Age Distribution: Good over Bad

Univariate Analysis

Univariate analysis focuses on the distribution and central tendency of a single variable. We will start by analyzing the numerical features, particularly: Credit Amount and Duration.

Here are some other statistics on the numerical features:

	Duration	Credit_amount
count	1000	1000
mean	20.90	3271.25
std	12.058	2822.73
min	4	250
25%	12	1365.5
50%	18	2319.5
75%	24	3972.25
max	72	18424

Insights:

As we can see from statistics, most of the credit amounts range between \$1,500 and \$4,000. From the first plot in Figure 1.5 we can see that most loans are for smaller amounts, creating a wide right dispersion for bigger quantities. Proportionally with the number of Credit Amount there's the duration of the loans (right plot) whose majority attests from 10 months to 24 months with 2 peaks in correspondence of the equivalent of 1 and 2 years. An even more specific graphical representations can be seen in Figure 1.6 and 1.7 distinguishing by good and bad.

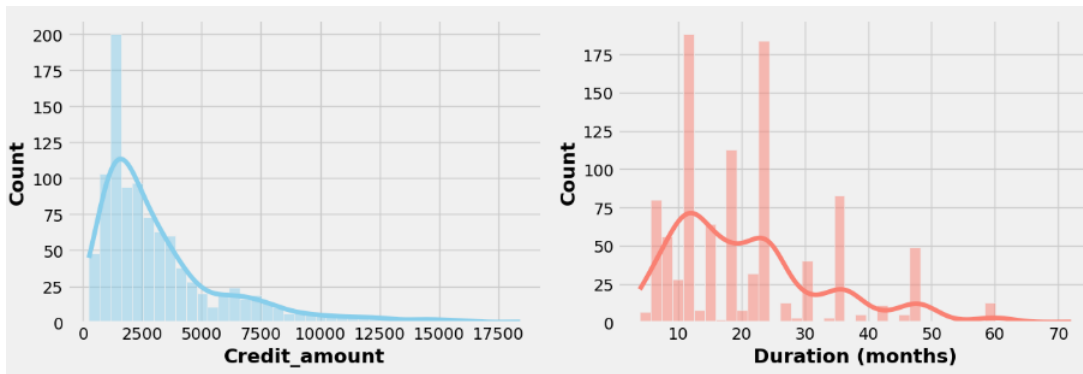


Figure 1.5: Credit Amount and Duration Distribution plots

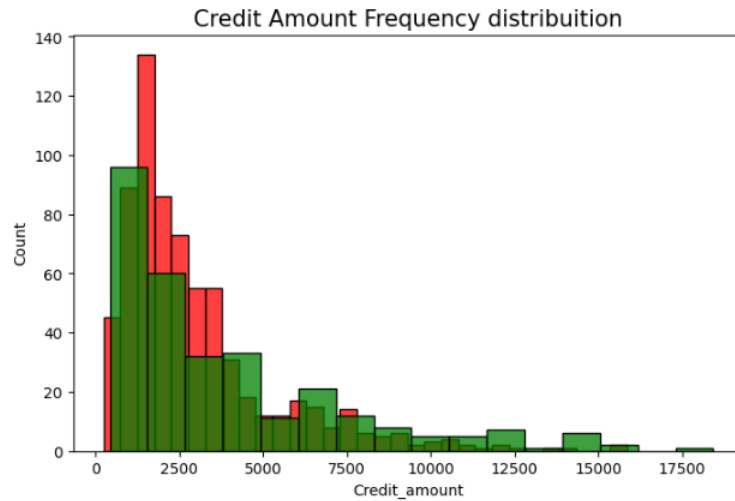


Figure 1.6: Credit amount by Good and Bad Distribution plot

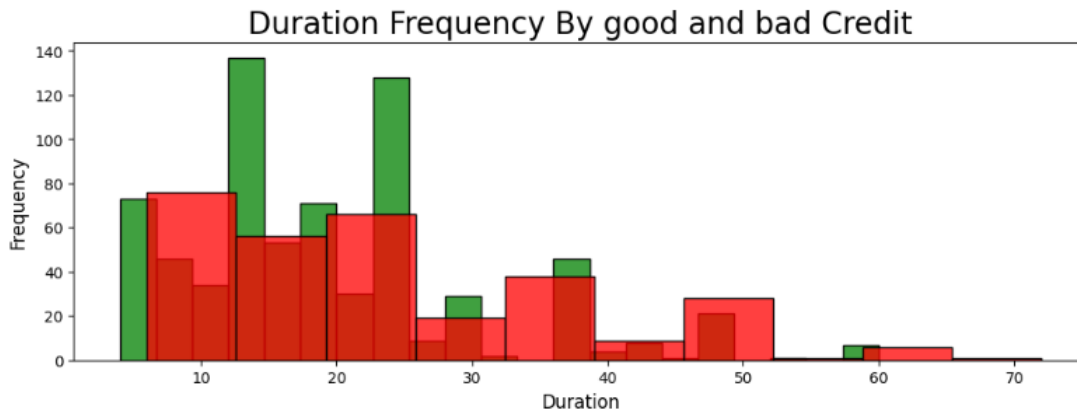


Figure 1.7: Duration by Good and Bad Distribution plot

Bivariate Analysis

After examining the main numerical features singularly we introduce Bivariate analysis that examines the relationships between two variables. Here, we investigate the correlation between Age, Credit Amount, and Duration.

Insights:

The Figure 1.8 shows the relationship between good and bad clients according to the numerical features expressed above.

- Credit Amount - Age:** From this plot is showed the relation between the amount of credit asked for the loan with the age of the customer. It immediately stands out that there's a huge increase in the interval of age starting from 50 with a decrease at 60 from Bad clients. This could be related to an economical policy of the bank that after a certain age the demand for a big amount of money from old people tends to be easily denied. This could be traduced to

the fact that, changing the age of the customer in a possible attack could deceive the system to evaluate it as a good candidate.

- **Duration - Age:** As in the first plot, all the customers with a venerable age get classified unsuitable for a long-term loan, underlying the importance of the age in the dataset (even if limited).
- **Credit Amount - Duration:** It is noticeable a linear relationship between the two features, suggesting that larger credits are typically associated with longer durations, making it easier for a customer to request a bigger amount of money decreasing the probability of being denied.

BIVARIATE ANALYSIS (HUE=RISK)

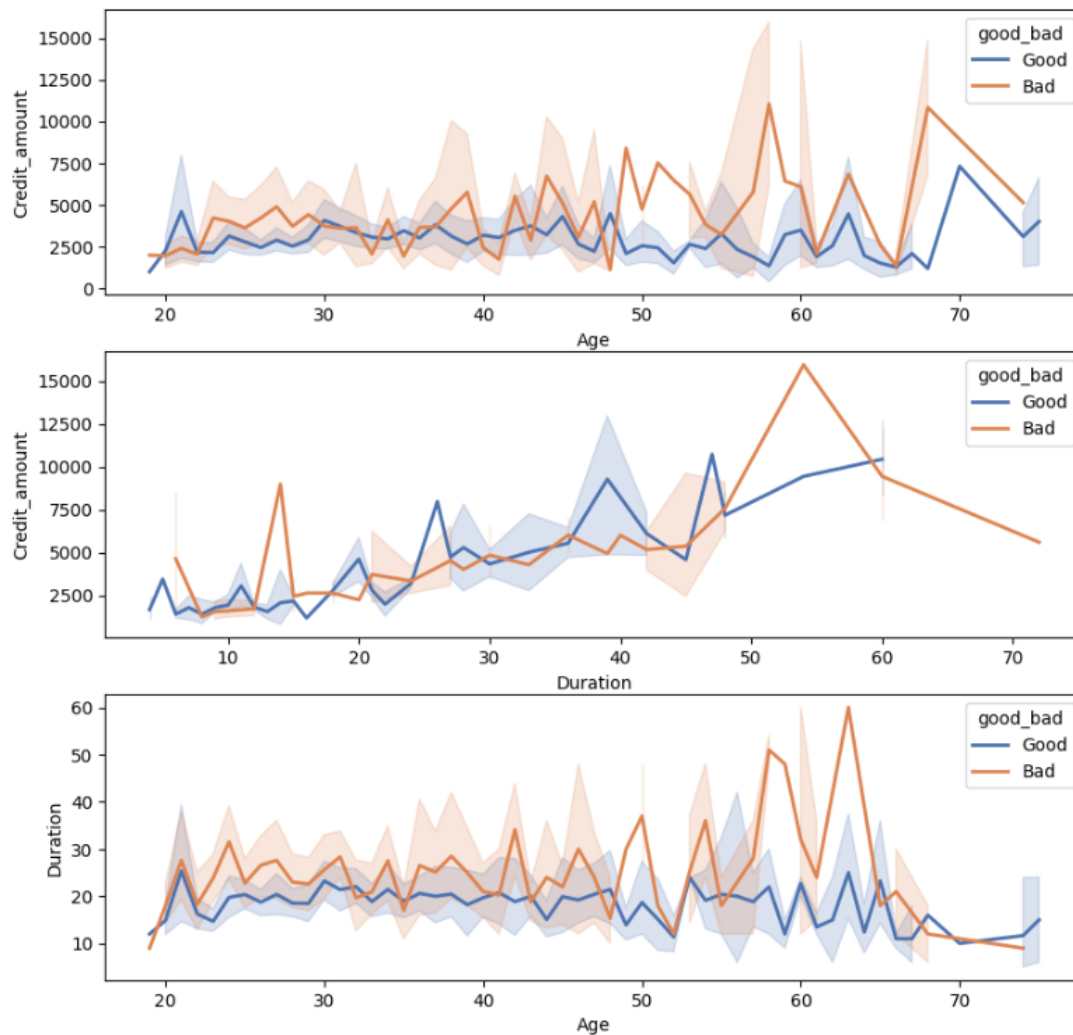


Figure 1.8: Bivariate analysis (Hue = Risk)

Pairplot Analysis: To better visualize potential linear relationships among features, we employ pair plots seen in Figure 1.9. These type of plots help identify correlations and patterns within the

dataset with multiple features at a time.

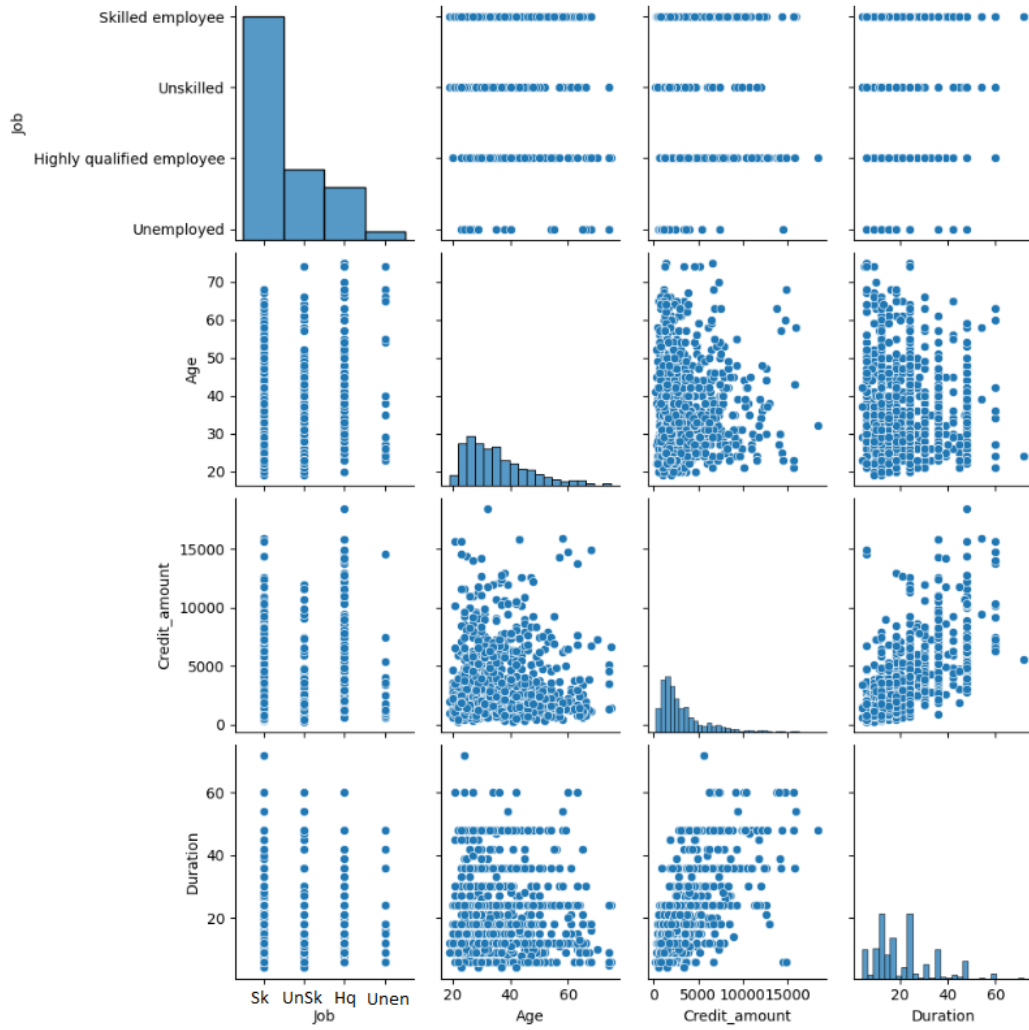


Figure 1.9: Pair Plot of Key Features

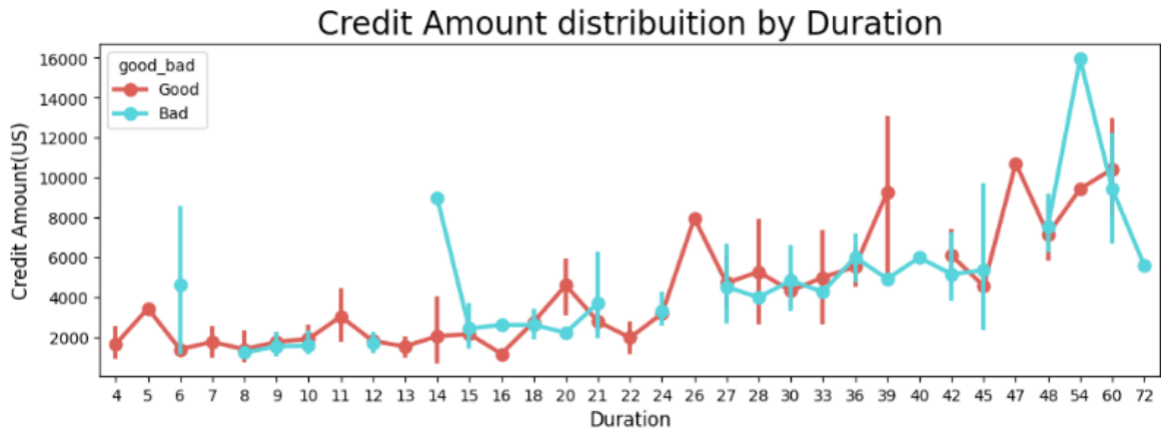


Figure 1.10: Relation analysis

While only hinted at by Figure 1.10, the linear relationship between Credit Amount and Duration

is even more clear from the pair plot, in which, there's a more dense area for low values and a more wide-spread area for increasing values. Let's try adding some enhancement for analyzing even further this correlation in more detail.

Enhancements:

In exploratory data analysis, "jitter" refers to adding a small amount of random noise to data points to prevent overlapping points from obscuring each other. This is particularly useful in visualizations like scatter plots or strip plots where many data points may have identical or similar values, causing them to overlap and making it difficult to see the distribution of the data.

An example of this technique can be seen in Figure 1.11.



Figure 1.11: Duration vs Credit (Jitter applied)

1.3.3 Categorical Variables

We explore the distribution of Credit Amount across different categories such as Housing, Job, Sex, Foreign Worker status, and Saving Accounts. Various visualizations are used to represent the Credit Amount distribution.

Note: For effective analysis and comparison, we convert the Age variable into categorical variables.

Job comparisons: The job categorical feature is composed by four different groups: Unemployed, Unskilled, Skilled, High-qualified Employee. Below are showed some plots to exalt a possible correlation with other numerical and categorical features.

- **Credit Amount and Job Comparison:** Figure 1.12 illustrates the relationship between these features, showing that highly qualified employees tend to get good classification for larger credit amounts more easily than other categories. However, despite this consideration, high qualified employee still face significant challenges. In fact, like the other categories,

they experience bad classifications when requesting relatively larger amounts. This might be an essential information which tells that the job feature has a limited impact over the classification, and possibly to perform any kind of attack would be better to consider other choices.

- **Age and Job comparison:** This comparison is shown in Figure 1.13.

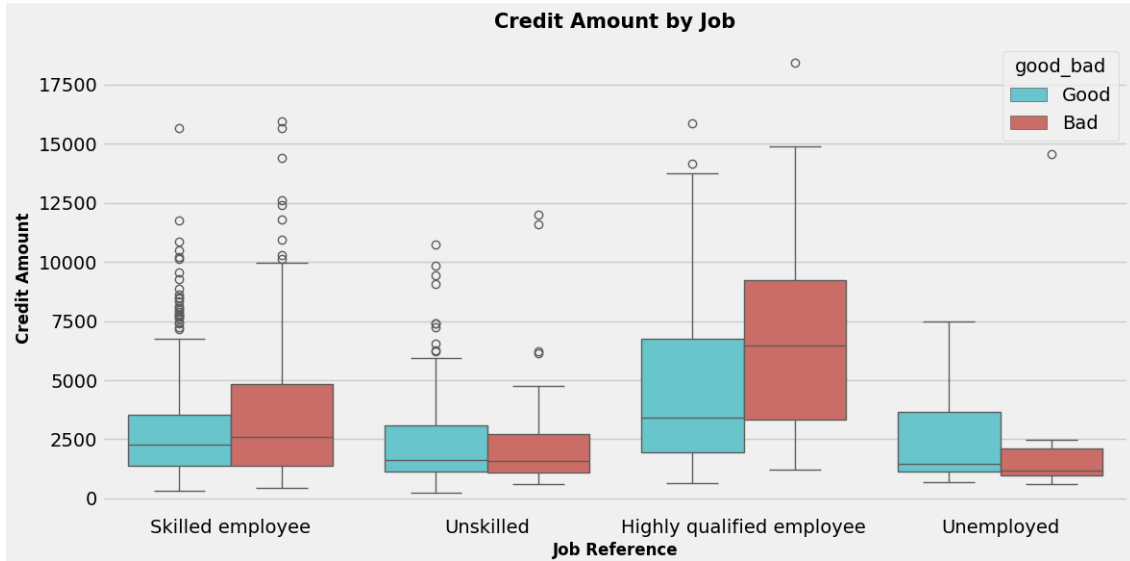


Figure 1.12: Credit amount and Job comparison

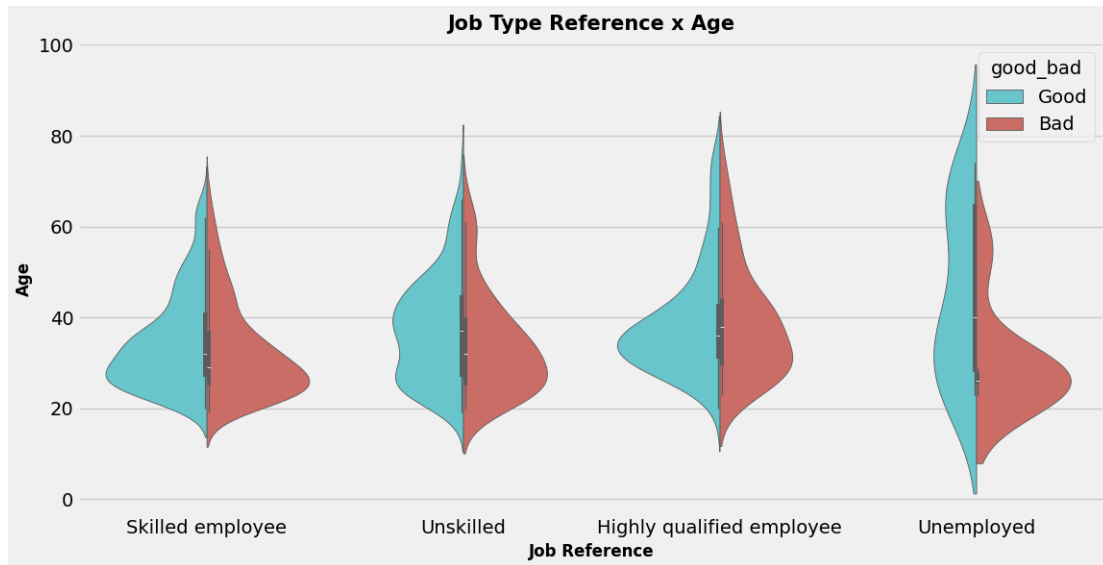


Figure 1.13: Age and Job comparison

Credit amount and Housing comparison: Housing feature is composed by 3 different classes: own, rent, free. Let's first analyze the distribution of this feature shown in Figure 1.14. The majority of the customers own a house. Notably, the relationship between credit classification (good and bad) and housing status differs among these groups. For "rent" and "free", bad credit cases exceed 50%

compared to good ones. While, for owners, bad credit is less than 40%. That implies an evident financial stability associated with owning a property.

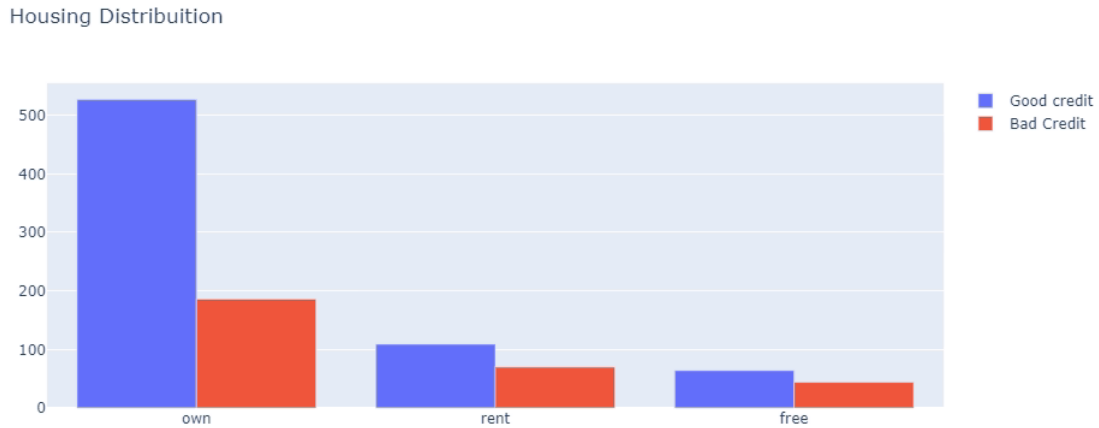


Figure 1.14: Housing distribution

In Figure 1.15 is shown the relationship between Credit Amount and Housing. Even though the major density resides in a low credit amount, it is visible a huge quantity of bad high-credits associated to free-housing clients.

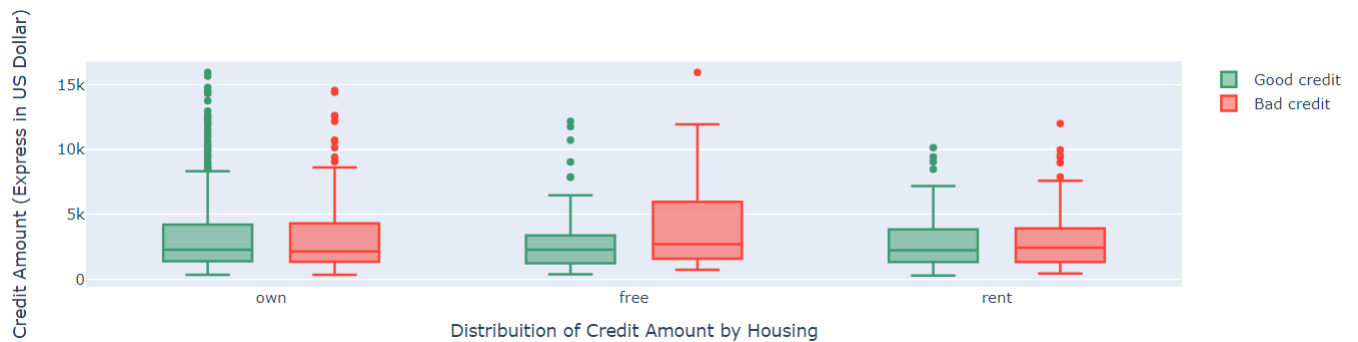


Figure 1.15: Credit amount and Housing comparison

Credit amount and Sex comparison: Before analyzing the relationship with credit amount, let's see the distribution of samples according to sex. In the left plot of Figure 1.16 we can see that the 70% of the total dataset is composed of male clients while the remaining 30% of females. Independently from the quantity, similarly as we saw in housing, in the case of males the percentage associated to bad credit is less than 50%, while for females is bigger. Comparing credit amount with sex we can see that there's almost an equality between the two sexes with a little disparity between good and bad.

Sex Distribution

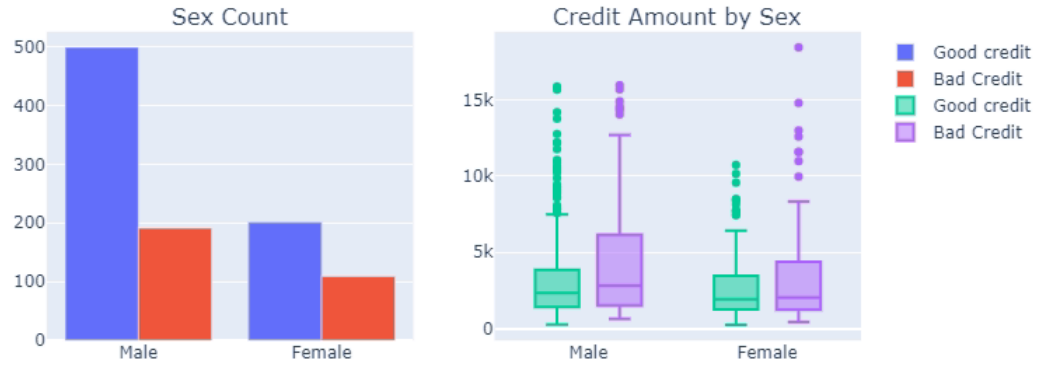


Figure 1.16: Credit amount and Sex comparison

Credit amount and Foreign Worker comparison: Analyzing Figure 1.17 we can see that the number of none-foreigners is extremely small compared with the number of foreigners (less than 5%). Therefore we consider irrelevant the dis-equality showed in the right plot, since the number of samples is inappropriate to draw conclusions.

Foreign worker Distribution

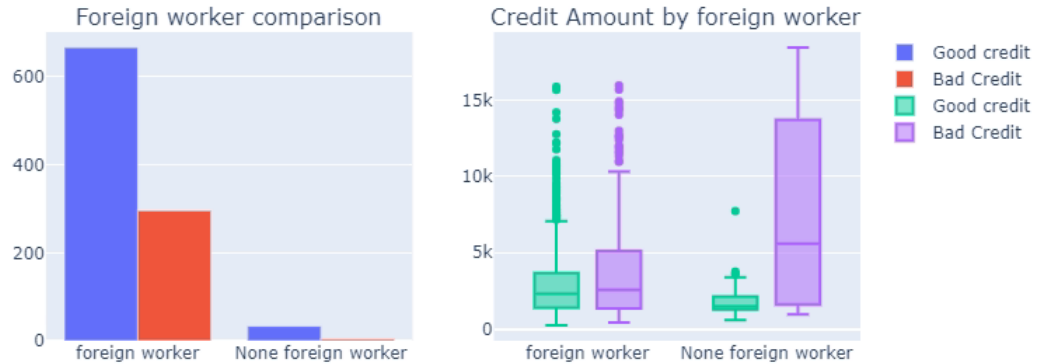


Figure 1.17: Credit amount and Foreign Worker comparison

Credit amount and Status Account comparison: Saving account feature is an essential component that represent the actual financial status of the client of the bank. It is composed by 4 main categories: normal, in debt, rich, unknown. Starting from Figure 1.18 we see the distribution of the clients. Even if, at first glance, the highest number of clients could seem unknown, taking in consideration both good and bad credit, the quantity of clients is almost uniformly distributed over the 3 classes "unknown", "normal" and "in debt". The most impressive information we retrieve

from Figure 1.19 is that in all cases the number of bad credit classifications are higher than the good ones, except for the "rich" category, where the number of good credit is bigger. This information could lead us to conclude that an important condition to increase the probability of getting good credit classification is to be on a high status account class.

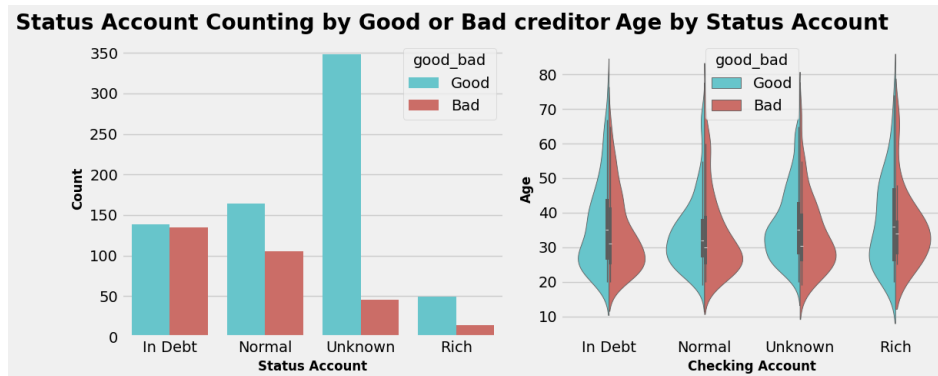


Figure 1.18: Status Account variable

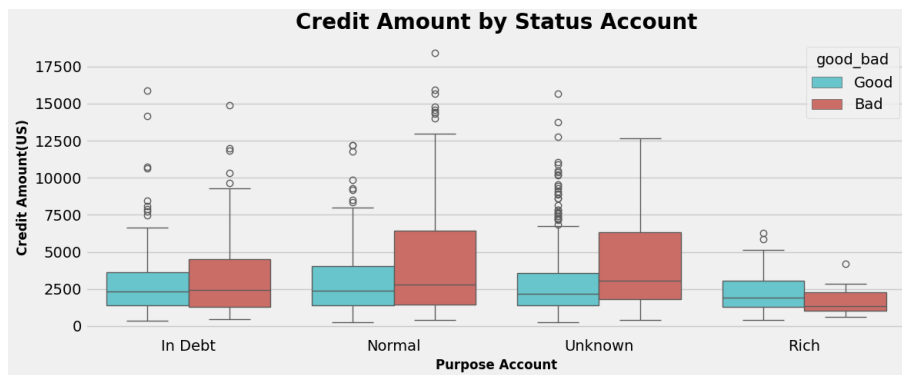


Figure 1.19: Status Account variable

1.3.4 Results

The relevant details extracted from this analysis include the correlation between Duration and Credit Amount, which we examined in detail using the jitter technique, and the significant disparity between good and bad creditors in the dataset. This disparity makes it challenging to determine which category is more relevant in terms of results. It is important to note that any observed imbalance cannot be specifically attributed to a particular category but rather to the overall imbalance between good and bad creditors within the database.

Chapter 2

Unsupervised exploration and clustering

2.1 Dimensionality Reduction For Data Visualization

In this chapter, we explore the main methods to reduce the number of attributes (columns) of our dataset, which is essential when dealing with a high-dimensional dataset. A large number of columns, indeed, can lead to various challenges, including:

- difficulty in extracting useful information due to the presence of irrelevant or noisy features
- difficulty in visualizing possible correlations among the attributes
- an increase in the computational time of all algorithms used, leading to higher waiting time for the training of our models

To address these issues, we used in particular 2 main techniques: PCA (Principal Component Analysis) and t-SNE (t-distributed Stochastic Neighbor Embedding):

2.2 PCA

PCA is used to evaluate the linear correlation between the attributes in a dataset and is able to create a new set of uncorrelated features which manages to maintain most of the original information contained in the dataset. In the new dataset the firsts features are constructed to maximize the information of initial data by capturing most of the variance of the original dataset. This process allows us to reduce the quantity of features while maintaining approximately the same quantity of information by representing the correlations between original features and the new ones that maximize the variance.

An important aspect of PCA is that the new features are uncorrelated because each of them is orthogonal to the other. The Figures [2.1](#) and [2.2](#) below show the correlation between the features of

the original dataset (before applying the PCA's reduction) and the correlation between the newly created features produced by the PCA.

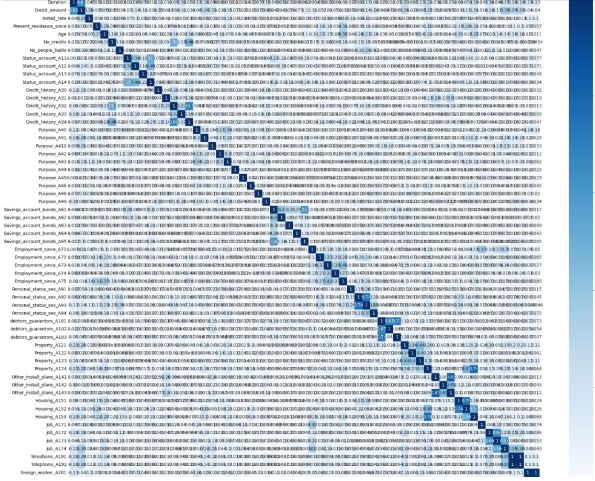


Figure 2.1: Correlation Matrix - before PCA

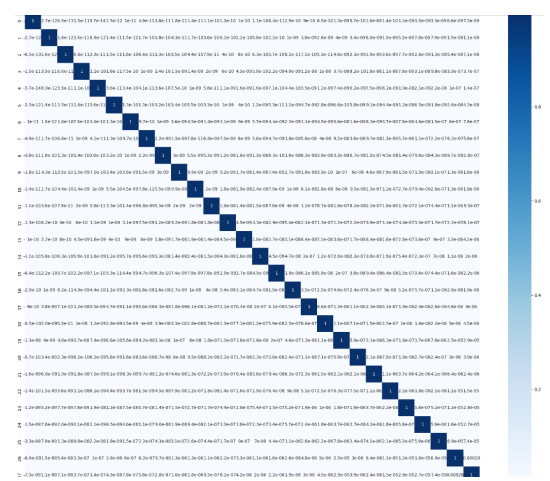


Figure 2.2: Correlation Matrix - after PCA

2.2.1 Theoretical foundation

Avoiding the majority of the theoretical aspects behind PCA, the main characteristics can be summarized as follows:

The covariance matrix S is a square matrix that contains:

- the linear relationship between the features outside the diagonal (covariances)
- the variances of the individual features on the diagonal

The goal of PCA is finding a new set of features that captures the maximum variance in the data with a lower dimensions. This is achieved by analyzing the covariance matrix (S) of the data.

The Covariance matrix can be computed as:

$$S = \frac{1}{m-1} (X - \bar{X})^T (X - \bar{X})$$

where:

- X : Attributes of the dataset
- $\bar{X}$: Matrix of the mean attributes
- m : Number of rows

To obtain the new features we have to find a base of vector, P , such that: $X' = PX$. According to linear algebra, the matrix P is the transpose of the matrix of eigenvectors. The eigenvectors represent the orthogonal direction that best describe the variance of data. By multiplying the original data with the transpose of P , we are projecting it along these new directions, obtaining new

features with maximized variance. These new features are the principal components we are looking for. In order to construct the eigenvectors we have to compute S from the original dataset and then resolve the characteristic equation ¹

$$|C - \lambda I| = 0 \quad (2.1)$$

where:

$$\lambda : \text{Eigenvalue of covariance matrix } S \quad (2.2)$$

2.2.2 Implementation

When dealing with our dataset we didn't have a large number of attributes such that PCA was an essential element. German-data, in fact, has a total of 21 features, which is not an excessive number when talking about features, especially with just 1000 samples. However, since the dataset contains a high number of categorical features, when performing dummy encoding technique the number grew up to 60, so we opted to perform PCA to see if there was the possibility to reduce the number of attributes while maintaining approximately the same performances.

For performing PCA we used `PCA()` function imported from python's scikit-learn library. The following code snippet shows how we fit the PCA model to our data:

```
pca = PCA(random_state=15)
pca.fit(X)
explained_variance = pca.explained_variance_ratio_
cum_exp_var = np.cumsum(explained_variance)
perc_cum_exp_var = cum_exp_var * 100
```

We evaluated the impact of dimensionality reduction on the data's variance by computing the cumulative explained variance increasing the number of features taken in consideration. This metric, plotted in Figure 2.3, helps determine which is the best trade-off between number of main components and original information.

¹Informations taken from https://en.wikipedia.org/wiki/Principal_component_analysis and from material supplied by the professor, in particular: 4_ML4N_DimensionalityReduction.pdf

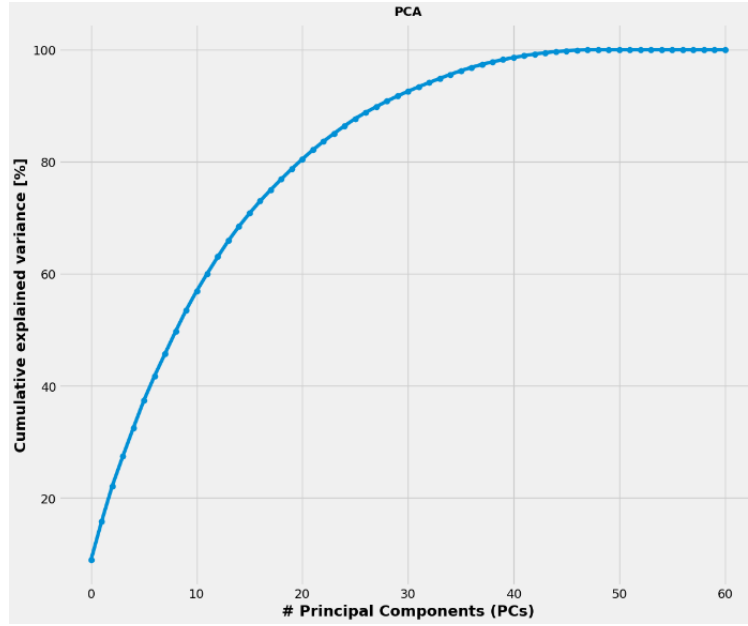


Figure 2.3: Explained cumulative variance %

Using the **Elbow Method**, it is clear from the plot that increasing the number of features after a certain threshold returns less and less information as the cumulative explained variance starts increasing very slowly until reaching 100% after the the 48th component. In order to further reduce the dimensions it's also possible to manually reduce the number of components at the cost of sacrificing a small percentage of variance. For this reason we decided to use 30 components having a cumulative explained variance of approximately 92%.

2.2.3 Evaluating PCA Component number through Sequential Removal and Performance Metrics

² Since our dataset has a limited number of samples, we conducted a test to verify the impact of varying the number of principal components on model accuracy using cross-validation over different models. This test could be an advantage to individuate the most convenient number of components but only for small datasets, nevertheless, it shouldn't be performed when the size of the dataset is huge because it would require a very long computation. We tested a range of components from 60 to 15, training five different classification models on each iteration: Decision Tree Classifier, Gaussian-NB, KNeighborClassifier, SVC and Logistic Regression³. After running the test we took the best values for each component iteration and plotted the accuracy scores. The results, visualized in Figure 2.4, indicate that, beyond the previously identified value of 48th component that maximize the variance, a further range of components offers an optimal trade-off between accuracy and computational efficiency. This range lies between 27 and 30 components. Within this interval, most

²We decided to insert the code relative to this section in a separate file (Adversarial Attack test PCA components) in order to not interfere with the code. This is a test that aims to find the optimal number of component so it should be ran only once. Furthermore we provided an already ran section where the plot 2.4 showed can be analyzed.

³See Chapter 3 for more details about supervised learning

of the classifiers stabilize on the same accuracy value (except from the Decision Tree). Beyond the 31th component, some classifier start increasing but others start significantly decreasing. Therefore, we could assert that choosing a number of components within the 27-30 range appears to be the most balanced choice considering both performance and computational time.

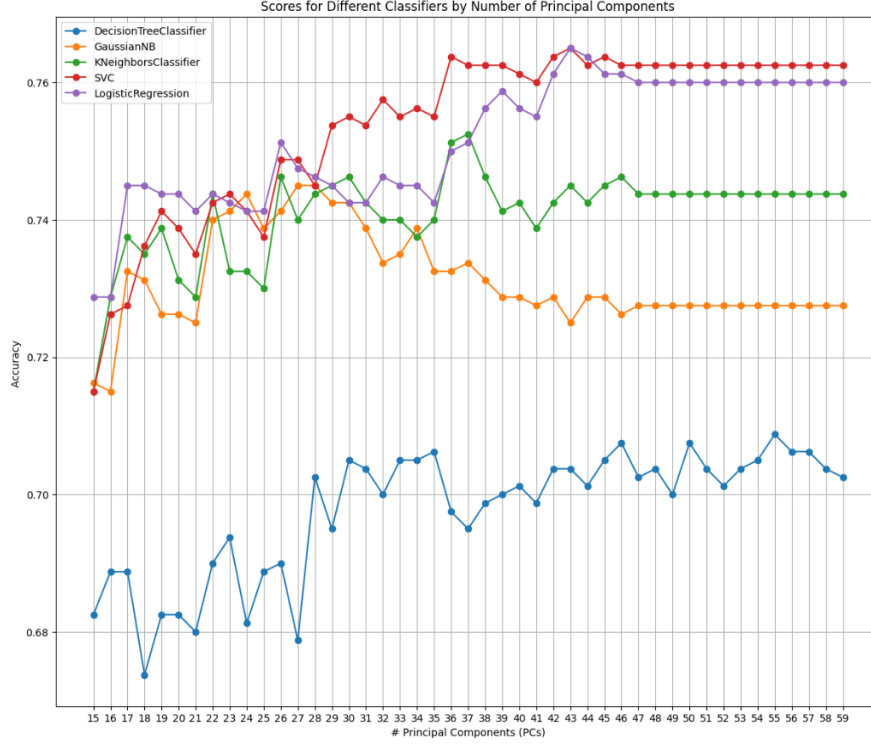


Figure 2.4: Accuracy of classifiers changing PCA components

2.3 t-SNE

2.3.1 Theoretical foundation

t-SNE (t-distributed stochastic neighbor embedding) is a non-linear dimensionality reduction technique used for datasets where the relationships between samples follow a non-linear pattern. The core objective of t-SNE relies on reducing the dimensionality of the dataset while preserving the structure of data. In particular, it has the capability of reducing the number of features to just 2 (or 3), giving the possibility of creating visual representations to easily visualize and identify patterns and clusters in the data. Another important aspect of t-SNE is its capability for anomaly detection. Since t-SNE keeps the local structure, similar data points are modeled as nearby points in the new structure, while dissimilar data points will be represented as distant points. This characteristic makes it easier to identify outliers/anomalies in the dataset as they will appear far from the main sets. We won't go in deeper mathematical details about how t-SNE works because it is beyond the scope of this discussion, however, we will just give the main idea of how it works.

- The algorithm starts constructing a probability distribution over all pairs of samples from the original dataset. In this distribution, as mentioned before, similar data points are assigned a high probability, while, dissimilar points are assigned a low probability.
- From this information, t-SNE defines a similar probability distribution over the points in the low-dimensional dataset to minimize the divergence between the high-dimensional data and low-dimensional probability distribution. ⁴

2.3.2 Implementation

For performing tSNE we used 'TSNE()' function imported from python's scikit-learn library. The following code snippet shows how we fit the t-SNE model to our data:

```
tsne = TSNE(n_components=2, perplexity=perplexity, random_state=42,
            learning_rate = lr)
X_tsne = tsne.fit_transform(X)
```

TSNE function accept in input 3 main parameters:

- **N. components:** To decide the final number of components of the output dataset
- **Perplexity:** used to emphasize relationships preservation between broader datapoints (high perplexity) or closer datapoints (low perplexity).
- **Learning rate:** used to control the step size take during the iterative optimization process of tSNE (the higher the faster it converges, but the less precise it is).

In our implementation of t-SNE, we explore various combinations of perplexity and learning rate to find the best suitable representation of our dataset. We tested 5 different values for both perplexity (ranging from 5 to 50) and learning rate (from 10 to 1000). The 'TSNE()' function initializes the model with the specific parameter combination, while, the 'fit_transform()' is in charge of fitting the model to our dataset, extracting the 2-dimensional representation. After the computation for each parameter combination, we generated the visual representation of the new datasets, assigning different colors to the points according to their label. Our initial expectation was that t-SNE would reveal clear separations or correlations between the two clusters, or at least exalt potential relationships between data. However, upon examining the plots, we did not observe any particular pattern or structure that could provide significant insights. Due to the substantial number of plots generated, and the absence of notable correlation, we opted not to include them in this report. However, they are available in the original code for reference.

⁴Informations taken from https://it.wikipedia.org/wiki/T-distributed_stochastic_neighbor_embedding and from material supplied by the professor, in particular: 4_ML4N_DimensionalityReduction.pdf

2.4 Unsupervised Data Analysis

In this section we will apply to our dataset two main families of unsupervised clustering algorithms: the Gaussian mixture model and the partitive algorithms (k-Means family), we decided to apply for each of them the two Dimensionality Reduction methods presented in the previous section namely PCA and t-SNE finally adding a third one not presented during the course but that we decided to experiment and deepen namely UMAP.

2.4.1 K-Means

K-Means is a widely used unsupervised clustering algorithm to divide a dataset into a predefined number of clusters, where each data point belongs to the cluster with the closest centroid. The main goal of K-Means is to minimize the sum of the squared distances between the data points and the centroid of their respective cluster, a measure commonly called inertia or SSE (Sum of Squared Errors).

The algorithm works through the following steps:

1. **Initialization of centroids:** Select k random points (or determine with other techniques such as `k-means++`), which work as initial centroids.
2. **Assigning Points to Centroids:** Each data point is assigned to the closest centroid.
3. **Calculating New Centroids:** The centroids are recalculated as the average of the points assigned to each cluster.
4. **Iteration:** Steps 2 and 3 are repeated until the centroids do not change significantly, or a maximum number of iterations is reached.

The K-Means algorithm requires the selection of some hyperparameters before being executed:

- **Number of Clusters (k):** This is the most critical hyperparameter, as it determines how many groups the data will be divided into. The choice of k can significantly affect the quality of the clustering.
- **Random State:** This is an integer value that controls the randomness of the centroid initialization. Using the same random state value allows for repeatable results between different runs of the algorithm.
- **Maximum Number of Iterations :** This indicates the maximum number of iterations that the algorithm will perform before stopping, even if it has not yet reached convergence.

Determining the Optimal Number of Clusters

To determine the optimal number of clusters we used several methods to try to get more hits in the hope of being more accurate.

Elbow method

The Elbow method is a graphical technique that allows us to identify the optimal number of clusters by observing the inertia as k varies. The idea is to choose the value of k at the point where the inertia begins to decrease less pronounced (forming an "elbow"). This indicates that adding more clusters does not bring significant improvements in error reduction.

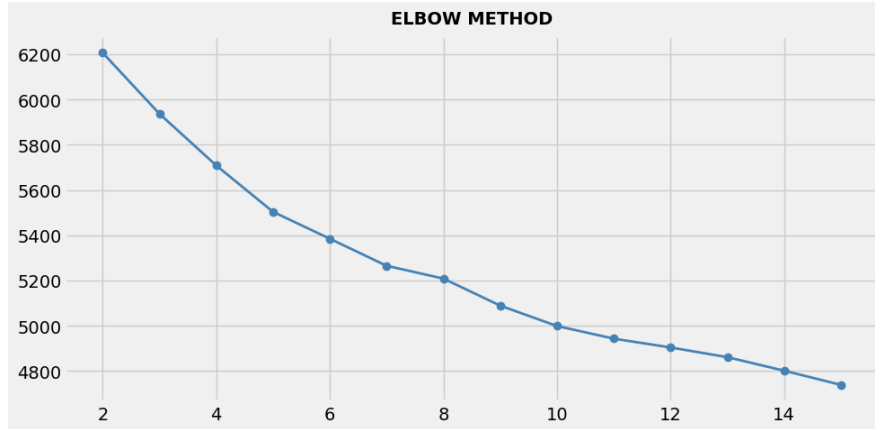


Figure 2.5: Elbow method

The graph is not conclusive as it does not show any specific evidence.

Silhouette Score

Silhouette score measures how well each data point is clustered with points in its own cluster compared to points in other clusters. A silhouette score value close to 1 indicates that the point is well assigned to its cluster, while a value close to -1 indicates a possible assignment error. The higher the average silhouette score, the better the clustering.

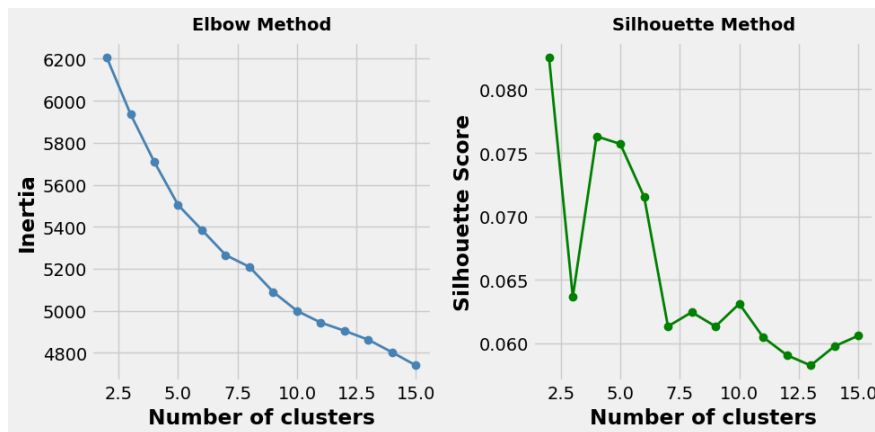


Figure 2.6: Silhouette Score vs Elbow method

In this case the Silhouette Score shows greater precision in point 2, that is, the one closest to the value 1. However, given the disorganized structure of the graph, we decide to continue to investigate this result further.

Random Sampling

Random Sampling allows to find the optimal number of clusters using a more robust approach, it

is a technique used to evaluate the stability and robustness of the results obtained by the K-Means algorithm with respect to the variability introduced by the random initialization of the centroids. When running K-Means, the algorithm chooses the initial centroids randomly, which can lead to different results in different runs, even on the same dataset. Once the results have been collected, they can be interpreted in several ways, in particular by looking at the distributions of the silhouette scores for each value of k. The first point to consider is the stability of the Silhouette Score: If the silhouette scores for a given k show low variability between runs, this suggests that the algorithm is stable for that number of clusters. Low variability indicates that K-Means converges to a similar solution even with different initializations, which is a positive sign. The second point is the average value of the Silhouette Score: Even if the algorithm is stable, the average value of the silhouette score for each k must be considered.

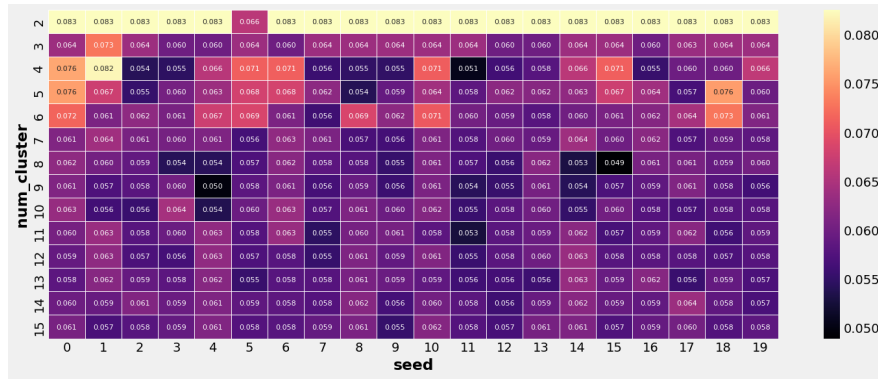


Figure 2.7: Random Sampling heatmap

In the heatmap, you can observe:

- **Colors:** The lighter colored cells indicate a better clustering as indicated in the legend. We then look for the value of k with the highest number of lighter cells.
- **Uniformity:** If for a given k the silhouette score values are uniformly high, this is a good indicator that k is a robust choice.

These factors lead us to confirm the observation previously made that two should be the optimal cluster value **Function for the Optimal Choice of k** To confirm that the optimal number of clusters is two, we implement a function that calculates the Silhouette score for different values of k and selects the k value that maximizes that score. The result confirms our previous assumptions. The function can be formulated as follows:

$$\text{Silhouette Score } k = \frac{1}{n} \sum_{i=1}^n \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

where:

- n is the number of data points.

- $a(i)$ is the average distance between point i and all other points within the cluster to which i belongs.
- $b(i)$ is the average distance between point i and all points in the nearest cluster to which i does not belong.

This section presents the results of a clustering analysis performed using k-Means combined with various dimensionality reduction techniques: PCA (Principal Component Analysis), PCA with SMOTE (Synthetic Minority Over-sampling Technique), t-SNE (t-Distributed Stochastic Neighbor Embedding), and UMAP (Uniform Manifold Approximation and Projection). The goal was to identify clusters in a dataset and evaluate the quality of these clusters using multiple metrics.

Methodology

1. **Dimensionality reduction:** The dataset was subjected to dimensionality reduction using PCA, SMOTE, t-SNE, and UMAP. These techniques help visualize high-dimensional data in a 2D space and potentially improve the separability of clusters.
2. **Clustering:** After dimensionality reduction, k-Means clustering was applied with the optimal number of clusters determined based on silhouette scores.
3. **Evaluation metrics:**
 - **Sum of squared distances (inertia):** measures how tightly the points are clustered around the centroid. Lower values indicate better-defined clusters.
 - **Silhouette score:** indicates how well-separated the clusters are. Scores close to 1 suggest well-defined clusters, while scores close to 0 suggest overlapping clusters.
 - **Rand index (RI) and adjusted Rand index (ARI):** measure the similarity between the clusters produced by k-Means and the actual class labels. RI ranges from 0 to 1, with higher values indicating better agreement. ARI adjusts RI by chance and can range from -1 to 1.
 - **Confusion matrix:** Compares the predicted clusters with the actual class labels.
 - **Precision, recall, and F1 score:** Provide information about the clustering accuracy with respect to the actual classes.

Result

Metodo	Inertia	Mean Inertia	Silhouette Score	RI	ARI	Accuracy	Precision	Recall	F1-Score
PCA	6206.27	6.21	0.083	0.5005	-0.0048	0.52	0.56	0.52	0.54
PCA + SMOTE	8237.32	5.88	0.085	0.5000	0.00005	0.51	0.51	0.51	0.51
t-SNE	975035.50	975.04	0.375	0.5015	-0.0006	0.47	0.58	0.47	0.49
UMAP	2610.14	2.61	0.480	0.5096	0.0053	0.57	0.60	0.57	0.57

Table 2.1: Comparison of different dimensionality reduction and clustering methods.

Interpretation of Results

- **Inertia:** UMAP produced the lowest inertia, suggesting that data points are more tightly clustered around their centroids than the other methods.
- **Silhouette Score:** UMAP also achieved the highest silhouette score (0.480), indicating better separation of clusters. t-SNE also provided a relatively high silhouette score (0.375), while PCA and PCA+SMOTE performed similarly with lower scores.
- **Rand Index and Adjusted Rand Index:** UMAP performed slightly better than other methods with an RI of 0.5096 and an ARI of 0.0053, suggesting better alignment with the actual class labels.
- **Confusion Matrix and Classification Metrics:** UMAP achieved higher precision, recall, and F1 score, indicating better classification of points into the correct clusters than the other methods.

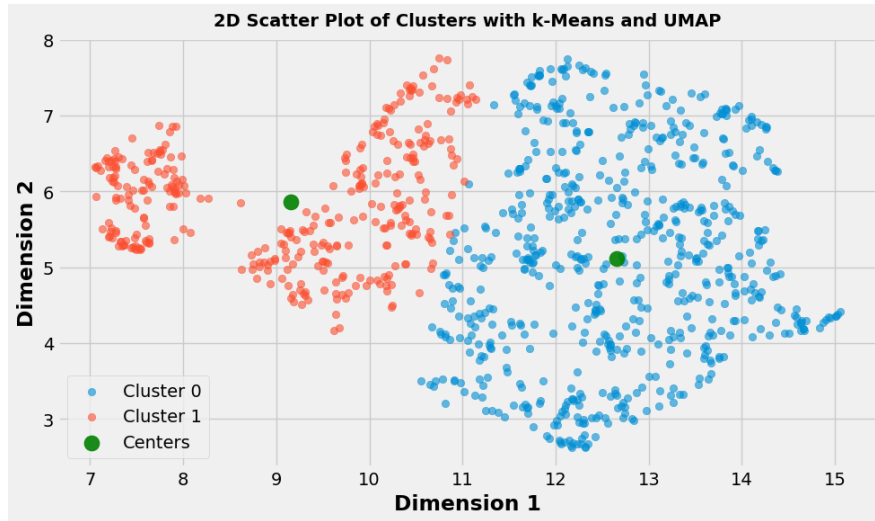


Figure 2.8: 2D Scatter Plot of Clusters with k-Means and UMAP

UMAP, combined with k-Means clustering, provided the best results in terms of inertia, silhouette score, and classification metrics. It effectively captured the underlying structure of the data and produced well-separated clusters, outperforming PCA, PCA with SMOTE, and t-SNE.

UMAP

UMAP (Uniform Manifold Approximation and Projection) is a dimensionality reduction technique that preserves more of the local structure of the data than PCA or t-SNE. It is particularly useful for clustering because it maintains the overall shape and relative distances of clusters in a lower-dimensional space, facilitating better separability. How UMAP Works:

- **Graph Construction:** UMAP starts by constructing a k-nearest neighbors graph of the data.
- **Graph Embedding:** It then optimizes the low-dimensional representation of this graph by minimizing the cross-entropy between the high-dimensional and low-dimensional representations.
- **Cluster Separation:** UMAP's flexibility in preserving local and global data structures makes it effective in distinguishing between clusters, leading to better-defined clusters for methods like k-Means.

2.4.2 Gaussian Mixture Model

The Gaussian Mixture Model (GMM) is a probabilistic model used for clustering data by assuming that the data is generated from a mixture of several Gaussian distributions. Each Gaussian distribution represents a cluster, and data points are assigned a probability of belonging to each cluster. This is a more flexible approach compared to k-Means, which assigns each data point to a single cluster with fixed centroids. The GMM was initially applied to the data without any dimensionality reduction. Several hyper-parameters were optimized, including the number of components (clusters), the type of covariance matrix, and regularization parameters. The best model was selected through grid search, which found that a full covariance matrix and a tolerance of 0.001 provided the best performance. Results:

- **Silhouette Score:** 0.083
- **Cluster Purity:** 58.% for Cluster 0 and 62.3% for Cluster 1
- **Confusion Matrix:** Indicates how well the clusters align with actual class labels, showing moderate alignment.

GMM with Dimensionality Reduction

- **PCA:** GMM was applied to data reduced to two principal components. The clustering results showed that GMM performed similarly to the previous application without dimensionality reduction, with a slightly improved silhouette score.
- **PCA + SMOTE:** PCA was combined with SMOTE to address class imbalance before applying GMM. This approach led to better cluster balance, but the silhouette score and cluster purity remained comparable to the results from PCA alone.
- **t-SNE:** GMM clustering on t-SNE-reduced data yielded a higher silhouette score compared to PCA-based results, indicating improved cluster separation. This method better preserved local structures in the data, which contributed to more meaningful clusters.
- **UMAP:** UMAP was used to reduce dimensionality and find the optimal number of clusters. GMM clustering on UMAP-reduced data achieved the highest silhouette score, indicating

the best clustering performance. This method effectively captured complex data structures, leading to the most accurate clusters.

Summary of Results

Method	Silhouette Score	Cluster Purity	Confusion Matrix Summary
GMM (No Reduction)	0.083	58.4% (Cluster 0), 62.3% (Cluster 1)	Moderate alignment with actual labels.
GMM + PCA	0.084	Similar to No Reduction	Similar alignment to No Reduction.
GMM + PCA + SMOTE	0.084	Similar to PCA	Better balance but similar to PCA results.
GMM + t-SNE	0.375	Moderate improvement	Improved cluster separation.
GMM + UMAP	0.441	Highest purity	Best performance with complex structures.

Table 2.2: Comparison of different dimensionality reduction and clustering methods.

- **Method:** The clustering method used.
- **Silhouette Score:** A measure of how well the data is separated into clusters.
- **Cluster Purity:** The purity of the clusters, i.e., the percentage of elements of a class that belong to each cluster.
- **Confusion Matrix Summary:** A summary of the confusion matrix, describing the alignment of the clusters with the true labels.

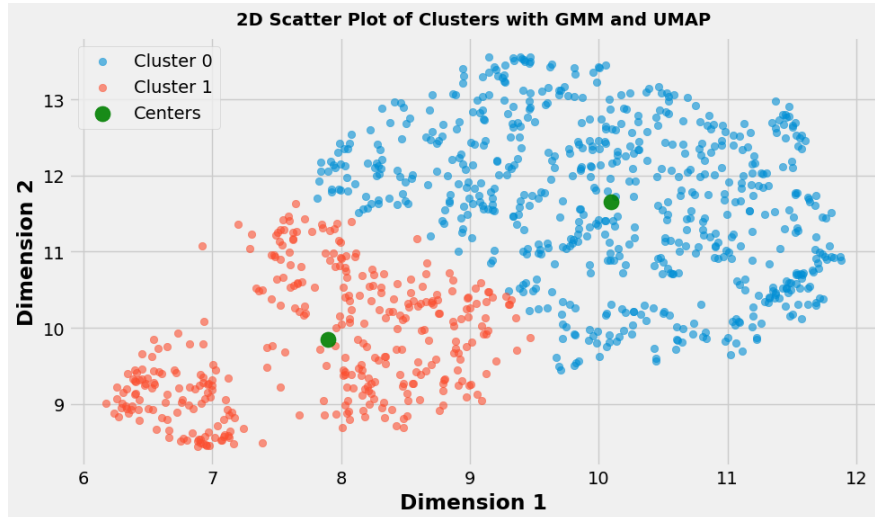


Figure 2.9: 2D Scatter Plot of Clusters with GMM and UMAP

GMM outperformed k-Means across all tested scenarios. While k-Means assigns each data point to a fixed cluster, GMM’s probabilistic approach provides a more nuanced view of the data. The higher silhouette scores and cluster purity observed with GMM suggest that it captures the underlying data structure more effectively than k-Means. GMM is particularly advantageous in handling clusters with complex shapes and varying densities, where k-Means might struggle due to its assumptions about cluster shapes and sizes.

Chapter 3

Supervised learning: classification

In this chapter we're going to look for the best classifier and respective hyperparameters for the classification task. We decided to split the cross-validation and subsequent evaluation, considering regular classifiers separately from a Neural Network model that we implemented.

3.1 Preamble

During the **Cross Validation** phase, and in particular in the subsequent evaluation on the test set, we encountered very low accuracy scores and very low class-specific metrics like **F1-Score and Precision**. Despite all the preprocessing steps taken to ensure the best possible model, we came to the conclusion that the original dataset suffered from a class imbalance problem: in particular the number of positive credit outcomes was too high with respect to the negative ones.

After trying a few solutions, we settled with the use of an **Oversampler** as a way to balance the classes: **SMOTE**.

3.1.1 SMOTE

SMOTE[2], which stands for Synthetic Minority Over-sampling Technique, is a method used to address the issue of class imbalance in datasets. SMOTE helps to balance the class distribution by generating synthetic samples of the minority class. Basically it first identifies the Minority Class (the class with less representation in the dataset), then it randomly selects an instance x_i from such minority class and finds its k-nearest neighbors (k-NN) from the same class. Among these k neighbors it selects one (x_{zi}) and creates a new *Synthetic Instance* through the following formula:

$$synthetic_instance = x_i + \lambda(x_{zi} - x_i) \quad (3.1)$$

where λ is a random number between 0 and 1 to ensure that the instance lies on the line segment between x_i and x_{zi} . This procedure is repeated for how many new instances we want to create.

A more in depth analysis into the differences between the dataset pre and post oversampling is

present in the *Adversarial Attack Supervised Data Analysis* file.

From a quick comparison between the confusion matrices from before (figure 3.1) and after (figure 3.2) the oversampling we can see a clear improvement, at least in the validation set.

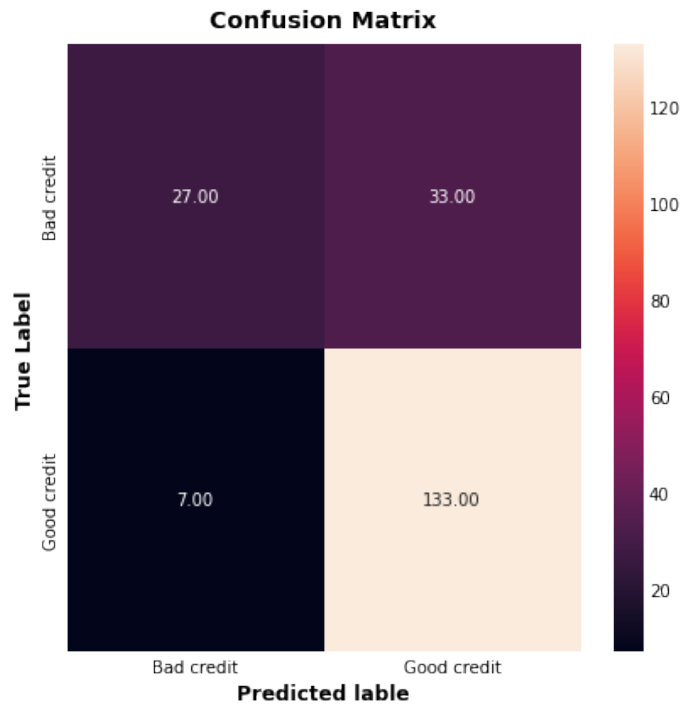


Figure 3.1: Confusion Matrix before SMOTE

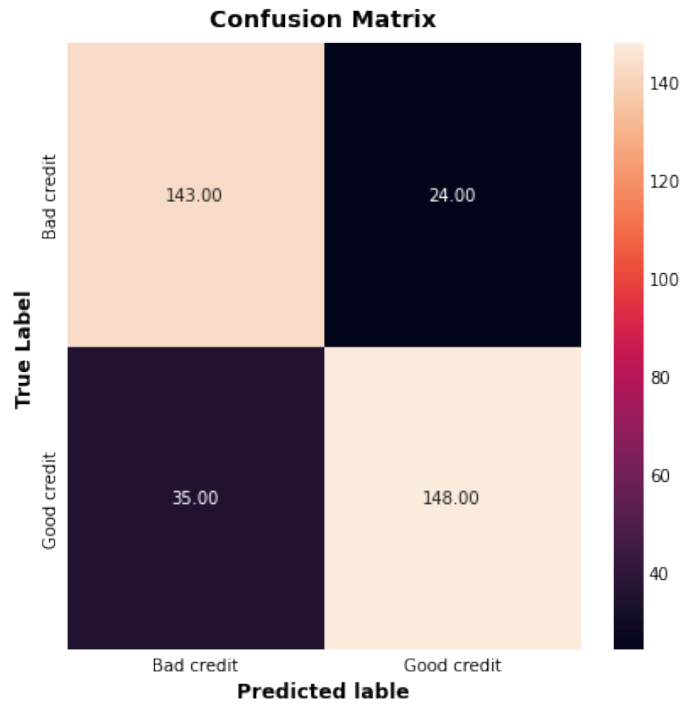


Figure 3.2: Confusion Matrix after SMOTE

3.2 Classifier Selection

We chose from various classifiers, notably:

- Decision Tree classifier
- Gaussian Naive Bayes classifier
- K-Nearest Neighbors classifier
- Support Vector Machines classifier
- Logistic regressor
- Neural Network

We performed the cross-validation process for each of these classifiers, evaluating their performance on the validation set after trying various combinations of **hyperparameters**.

For all non-Neural Networks classifiers we employed the ***GridSearchCV*** function of ***Scikit-Learn***. The list of hyperparameters was taken directly from the documentation of Scikit-Learn[1] and the values on which the validation was performed taken from both examples on the website and from the laboratories performed during the course.

What follows is a description of the classifiers and their main hyperparameters considered for the **Grid Search**.

3.3 Cross Validation

3.3.1 Here we consider non NN classifiers

Decision Tree Classifier

This is a model used for classification tasks. It predicts the class label of an instance by navigating through a series of decision nodes, from the root of the tree down to a leaf node. Each internal node represents a "test" on an attribute, each branch represents the outcome of the test, and each leaf node represents a class label (the decision taken after computing all attributes).

It has various steps:

- **Node Selection:** Starting from the root node, the tree splits the data based on an attribute that results in either the highest information gain or another criterion.
- **Splitting Criteria:** Various metrics are used to determine the best split, such as Gini impurity or information gain (entropy or log_loss).
- **Tree Growth:** This process is repeated recursively for each child node, creating branches and further splits.

- **Stopping Criteria:** The tree growth stops when a stopping criterion is met, such as maximum depth, minimum samples per node, or if no further improvement can be made.

The hyperparameters we chose to evaluate and tune were:

- **Criterion:**
 - Gini: Uses the "Gini" impurity. Used by default in the DecisionTreeClassifier of scikit-learn.
 - Entropy or Log Loss: Measures the Shannon information gain (a measure from information theory).
- **Max Depth:** The maximum depth of the tree. This prevents the tree from growing too complex and overfitting the data.
- **Min Samples Split:** The minimum number of samples required to split an internal node.

```
Best Score for DecisionTreeClassifier: 0.7660714285714285
Best Parameters {'criterion': 'entropy', 'max_depth': None, 'min_samples_split': 4}
```

Figure 3.3: Results for Decision Tree Grid Search

In 3.3 we can see the set of best hyperparameters and the accuracy score found by the *GridSearchCV*, function of *Scikit-Learn*.

Gaussian Naive Bayes Classifier

The Gaussian Naive Bayes (GNB) classifier is a probabilistic machine learning model based on Bayes' theorem, assuming that the features follow a Gaussian (normal) distribution. It's a type of Naive Bayes classifier specifically suited for continuous data. It uses Bayes' theorem to calculate the probability of a class given a feature set. It assumes that the likelihood of the features given a class follows a Gaussian distribution.

It's a very fast and simple classifier, but the independence and Gaussian assumptions that it makes regarding the dataset make it less powerful than other more complex classifiers.

Its only hyperparameter is *Var_smoothing* (default 1×10^{-9}) which adds a small value to the variance of each feature to prevent division by zero in case of zero variance in the training data. It is a form of regularization.

```
Best Score for GaussianNB: 0.7133928571428572
Best Parameters {'var_smoothing': 0.0001}
```

Figure 3.4: Results for Gaussian Naive Bayes Grid Search

In 3.4 we can see the best *var_smoothing* value and the accuracy score found by the *GridSearchCV*. The accuracy is the lowest among our classifiers, we think that it's because of the assumption made by the classifier on the nature of the feature distribution which is clearly not a Gaussian distribution.

K-Nearest Neighbors Classifier

The k-nearest neighbors (k-NN) classifier is a simple, non-parametric, instance-based learning algorithm used for both classification and regression tasks. In classification, it assigns a class label to a query point based on the majority class among its k nearest neighbors.

It's still a relatively simple model to implement but makes no assumption on the data distribution and is more computational heavy.

1. The algorithm calculates the distance between the query point and all points in the training set using a specified distance metric, commonly Euclidean distance.
2. It identifies the k closest points (neighbors) to the query point based on the calculated distances.
3. The class label of the query point is determined by the majority class among these k neighbors. In case of a tie, various strategies can be applied, such as choosing the class of the nearest neighbor.

The hyperparameters we chose to evaluate and tune were:

- **Number of Neighbors**
- **Weights:** determines how to weight the contributions of the neighbors
 - Uniform: All neighbors contribute equally to the decision.
 - Distance: Closer neighbors contribute more significantly.
- **Algorithm:** The algorithm used to compute the nearest neighbors
 - Auto: Automatically selects the best algorithm based on the values passed to fit method.
 - Ball Tree: Uses BallTree algorithm.
 - Kd Tree: Uses KDTree algorithm.
 - Brute: Uses brute-force search

```
Best Score for KNeighborsClassifier: 0.85625
Best Parameters {'algorithm': 'auto', 'n_neighbors': 2, 'weights': 'uniform'}
```

Figure 3.5: Results for Nearest Neighbors Classifier Grid Search

In 3.5 we can see the best hyperparameters and the accuracy score found by the grid search. The accuracy is pretty high, in particular we see the the best number of neighbors was 1, this gives us a

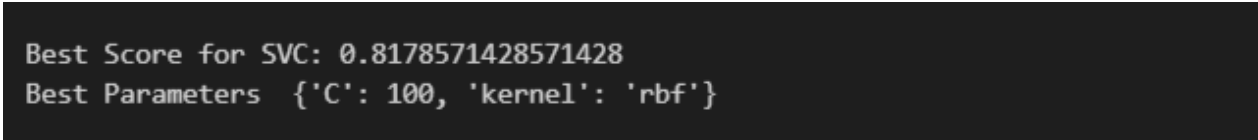
model very prone to **overfit** and possibly very **vulnerable to noise** as it places a lot of importance to each single point. This means that a slight change or mislabeling may heavily impact the accuracy of the model.

Support Vector Machines Classifier

Support Vector Machines (SVM) are a set of supervised learning methods used for classification, regression, and outlier detection. They are particularly effective in high-dimensional spaces and are versatile due to the use of different kernel functions.

In particular here we employed C-Support Vector Classification, where the basic idea is to find a hyperplane that best separates the data points of different classes. The optimal hyperplane is the one that maximizes the margin, which is the distance between the closest data points (called support vectors) of different classes. The hyperparameters that we tuned were:

- **C**: Controls the trade-off between achieving a low error on the training data and minimizing the norm of the weights. A smaller value of C makes the decision surface smooth, while a larger value of C aims to classify all training examples correctly.
- **Kernel**: Specifies the kernel type to be used in the algorithm
 - 'linear': Linear kernel
 - 'poly': Polynomial kernel
 - 'rbf': Radial basis function (Gaussian) kernel
 - 'sigmoid': Sigmoid kernel.



```
Best Score for SVC: 0.8178571428571428
Best Parameters {'C': 100, 'kernel': 'rbf'}
```

Figure 3.6: Results for Support Vector Machines Grid Search

In 3.6 we can see the results for the SVM(SVC) grid search. The accuracy was (on average) the highest among our classifiers, we think that it's due to the small size of the dataset and also because it's a very resilient model against overfitting.

Logistic Regressor

Logistic regression is a widely used statistical method for binary classification tasks. It predicts the probability that a given input belongs to a particular class. Despite its name, logistic regression is a linear model for classification rather than regression. It models the relationship between a set of input features X and a binary outcome y using a logistic function (also known as the **sigmoid**

function). The decision boundary is determined by the threshold applied to the predicted probabilities. If the predicted probability is greater than the threshold, the instance is classified as the positive class; otherwise, it is classified as the negative class.

The tuned hyperparameters were:

- **Penalty:** to specify the norm used in the penalization
 - 'l1': L1 regularization (Lasso).
 - 'l2': L2 regularization (Ridge).
 - 'l1': Combination of l1 and l2.
 - 'none': No regularization.
- **Solver:** it's the algorithm used in the optimization problem
 - 'newton-cg': Uses Newton's conjugate gradient method.
 - 'lbfgs': Uses Limited-memory Broyden–Fletcher–Goldfarb–Shanno (BFGS).
 - 'liblinear': Uses a coordinate descent algorithm (good for small datasets).
 - 'sag': Stochastic Average Gradient Descent.
 - 'saga': Variant of SAG that supports L1 regularization.
- **C:** Inverse of regularization strength, smaller values indicate stronger regularization
- **Max Iterations:** simply the maximum number of iterations after which the solver stops if it hasn't yet converged.

This model is quite simple as it is essentially a linear regressor which employs a different loss function. This makes it quite a fast model (especially using a low number of max iterations) but one which assumes a linear relationship between the features and the log-odds of the outcome.

```
Best Score for LogisticRegression: 0.7401785714285715
Best Parameters {'C': 0.1, 'max_iter': 200, 'penalty': 'l2', 'solver': 'lbfgs'}
```

Figure 3.7: Results for Logistic Regressor Grid Search

In 3.7 we can see the results of the grid search, which show a quite low score for the accuracy metric. Despite previously scaling the entire dataset (basically almost a requirement to have decent result with this classifier), we still witnessed a low result when compared with the other classifiers.

3.3.2 Neural Network classifier

To evaluate the Neural Network's behavior, particularly under simulated adversarial attacks, we conducted multiple experiments with different configurations, including variations with and without PCA and SMOTE. After thorough testing, we observed that, on average, the Neural Network

classifier performed better when neither PCA nor SMOTE was applied. Consequently, we decided to use the original dataset without any modifications for our final assessments.

The neural network model we implemented is composed of an input layer, three hidden layers, and an output layer. Each hidden layer uses the ReLU activation function to introduce non-linearity.

For the training, the dataset was split into training and validation sets, with 85% allocated for training and 15% for validation. This split ensures the model generalizes well and isn't overfitting to the training data. The model was implemented using PyTorch, where a custom class "PyTorch-Classifier" was created to handle training, prediction and evaluation processes.

During training, we tracked model's loss and then we plot a validation curve to monitor the performance over epochs. This helped in identifying whether the model was converging and when it started to overfit.

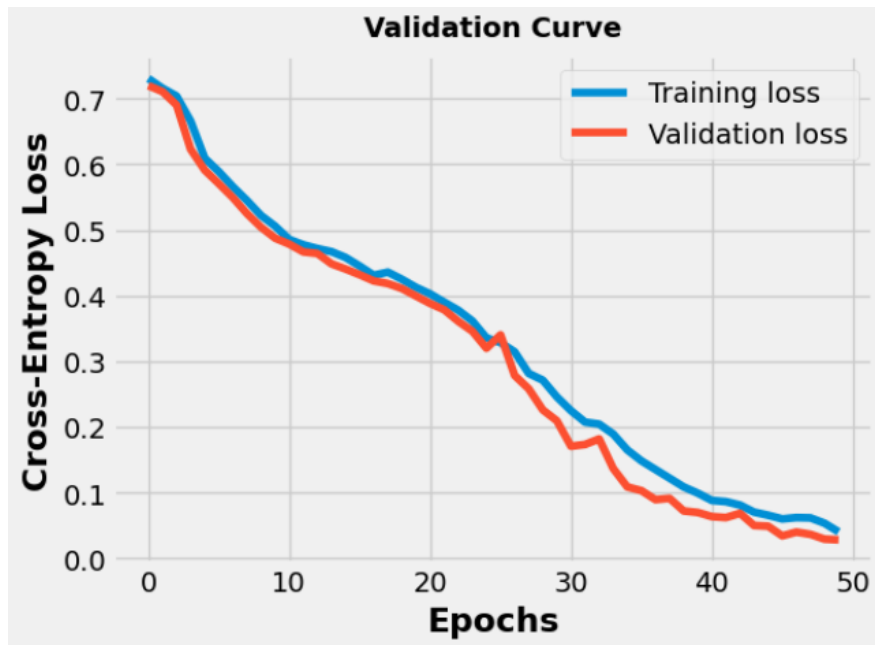


Figure 3.8: Validation Curve NN with PCA

Hyperparameter Tuning

Hyperparameter tuning was conducted using a grid search approach, exploring various configurations, including the number of neurons in each hidden layer, learning rate, batch size and the number of epochs. The grid search was implemented using "GridSearchCV" from the scikit-learn library, combined with the "StratifiedKFold" method to maintain the distribution of classes across the folds. For each combination of hyperparameters, the model was trained, and its performances evaluated according to accuracy metric on the validation split. Then we saved the configuration that led to the highest validation accuracy as the best model.

3.4 Classifier Evaluation

3.4.1 Non-NN Classifier

We ended up choosing the SVM classifier since it was the one which obtained the highest accuracy score. In particular we can see from 3.9 and 3.10 the values of the accuracy scores depending on the hyperparameters values, where it's clear that the better performing **kernel** and **C** choices were respectively *"rbf"* and *"100"* which are the hyperparameters values we ended up using.

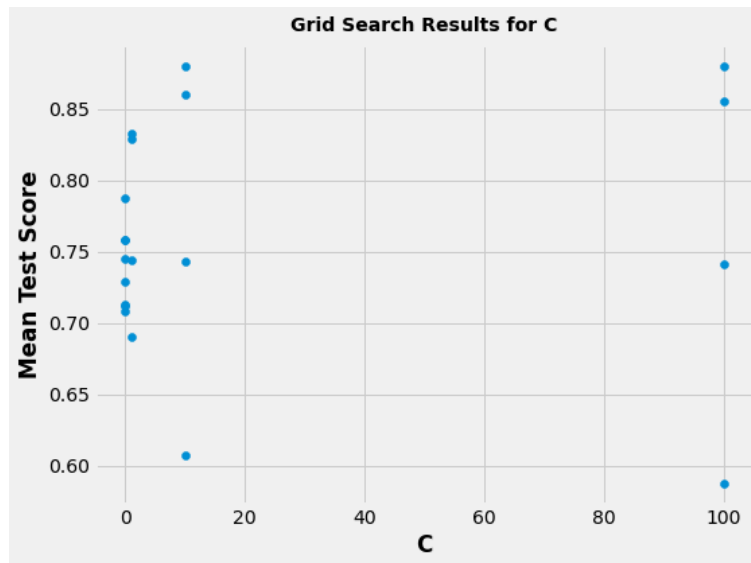


Figure 3.9: Accuracy scores of SVM based on C value set

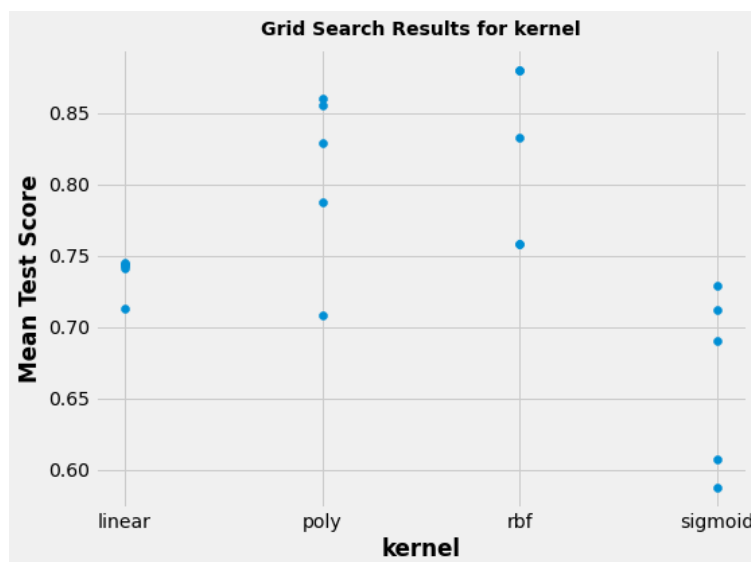


Figure 3.10: Accuracy scores of SVM based on Kernel value set

We then proceeded to refit the model with the chosen hyperparameters and perform the prediction and subsequent evaluation on the Test Set previously created.

Using classifier : SVC()					
	precision	recall	f1-score	support	
0	0.50	0.50	0.50	60	
1	0.79	0.79	0.79	140	
accuracy			0.70	200	
macro avg	0.64	0.64	0.64	200	
weighted avg	0.70	0.70	0.70	200	

Figure 3.11: Classification Report of SVM

The results shown in 3.11 are not exceptional, while we maintain a decent accuracy score (pretty much in line with some weaker models) it's still quite low when compared with the one obtained in the validation phase. But the real problem of the model is its inability to consistently classify negative credit labels correctly as shown in the confusion matrix in 3.11 and the **ROC Curve** in 3.13, where the **AUC** value is just 0.61: dangerously close to 0.5, which is just random guessing.

Even after the use of SMOTE to rebalance the class, the model still seems to privilege the positive class over the negative one. This led us to believe that the model was **overfitting** since there was a very big gap in performance between the scores on the validation set and the test set. We attributed this issue to the small size of the dataset which didn't allow the model to correctly "learn" the patterns to recognize the negative labels.

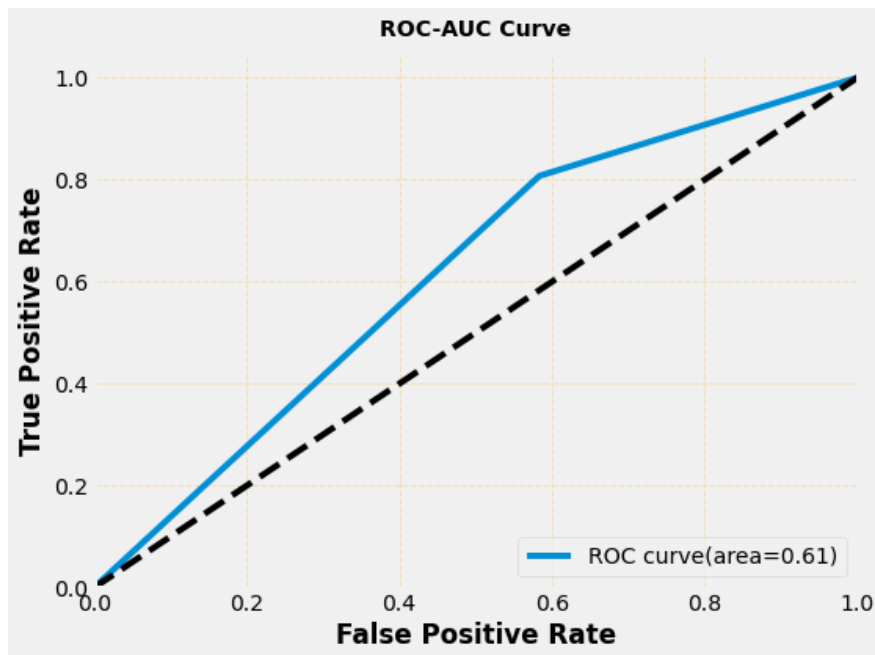


Figure 3.13: ROC Curve of SVM

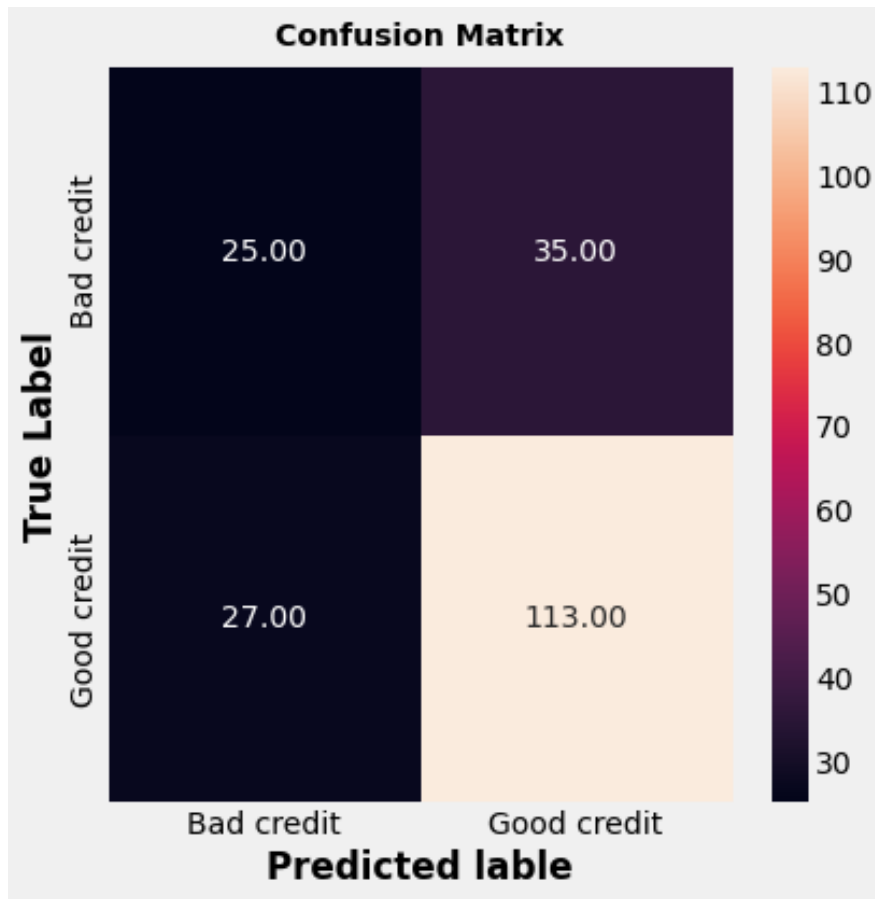


Figure 3.12: Confusion Matrix of SVM

3.4.2 Neural Network classifier

After selecting the best hyperparameters, the final model was evaluated on the test set, generating a confusion matrix and a detailed classification report.

Results

After the training process, the model achieved a validation accuracy of 80 and the accuracy in the test set reached 84%. As shown by the confusion matrices in Figure 3.14 and Figure 3.15, the model has some difficulties predicting true positives. This issue can be attributed to the low quantity of data in the dataset or in the low complexity of the Neural Network model. Below is shown the classification report of the final Neural Network model:

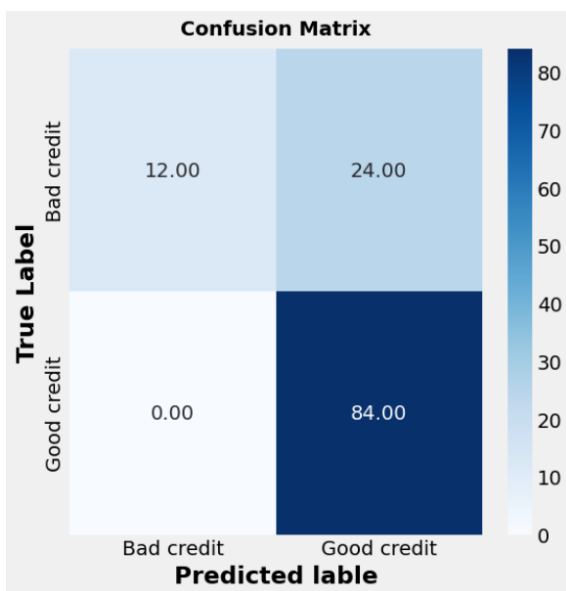


Figure 3.14: Confusion Matrix Validation set NN

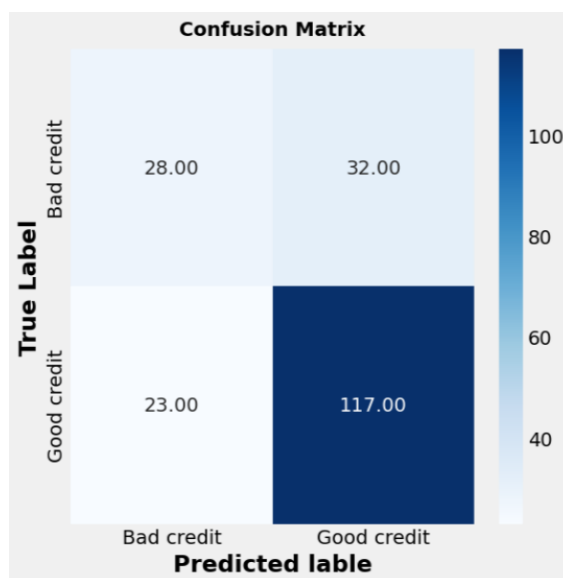


Figure 3.15: Confusion Matrix Test set NN

	precision	recall	f1-score	support
0	0.55	0.47	0.50	60
1	0.79	0.84	0.81	140
accuracy			0.72	200
macro avg	0.67	0.65	0.66	200
weighted avg	0.71	0.72	0.72	200

Figure 3.16: Classification report NN Test set

Chapter 4

Adversarial Attack

This section aims to evaluate the robustness of our model against potential adversarial attacks on the analyzed dataset. We expect to observe some performances degradation ¹ on predicting labels while, at the same time, aiming to develop a mechanism to prevent or defend from such attacks. In our analysis to simulate an adversarial attack we generate normally distributed noise with varying magnitudes in each iteration to test the resistance of classifiers.

4.1 Random Noise

This section examines the impact of random noise applied over the test set, measuring its effect on the classifier performances. We aim to determine the level of degradation in performance achieved by changing the intensity of the noise according to the noise level parameter ε (epsilon). Due to the necessity of using a neural network classifier for subsequent analyses, we decided to use it also for this case.

To simulate a gradually escalating attack, we introduced noise with variable intensity ϵ ranging from 0.5 to 20. In the first iteration, the added noise is minimal, but as the simulation progresses, the noise intensity increases significantly. The noise is drawn from a normal distribution, scaled by $\epsilon/\sqrt{d}$, where d is the number of features, and clipped to ensure the added noise remains within the range $[-0.5, 0.5]$. This ensures that even at the highest noise levels, the modified data remains comparable to the original. The evaluation was repeated 10 times for each noise level, and the average balanced accuracy score was recorded.

- Noise function:

Here the noise is generated and added to the test set. We randomly generate a set of values from a normal Gaussian distribution with mean $\mu=0$. This means that, theoretically, all the values from $-\infty$ to $+\infty$ could appear. However, most of the values will fall within a relatively small range of 0. Furthermore, in order to iteratively increase the intensity of the noise we used

¹To measure performance we will take in consideration accuracy (in this case considered "Balanced Accuracy").

the "scale" parameter that allows to set the standard deviation of the distribution, then, after the generation of the noise vector, we computed the norm and clipped the noise it according to the ε value using `np.clip()`. In this analysis must be taken in consideration the cases where the values of each attribute will exceed the interval $[0,1]$. In order to retain the true impact of the noise on the classifier we decided not to standardize the data. By doing so, we can more accurately measure how much the classifier is robust to noise, taking in consideration that in real-world scenarios, adversarial perturbations might produce out-of-distribution samples.

Finally we evaluate the performance of the classifier and then plot the metric (balanced accuracy) varying the value of noise level (ε).

4.1.1 Performance Analysis

At each iteration we computed the confusion matrix to show the performances of the classifier in presence of gradual noise. Due to the large number of images, we'll focus on the most significant ones: Figure 4.1 (Noise Level $\varepsilon = 0.5$) and Figure 4.2 (Noise Level $\varepsilon = 19.5$).

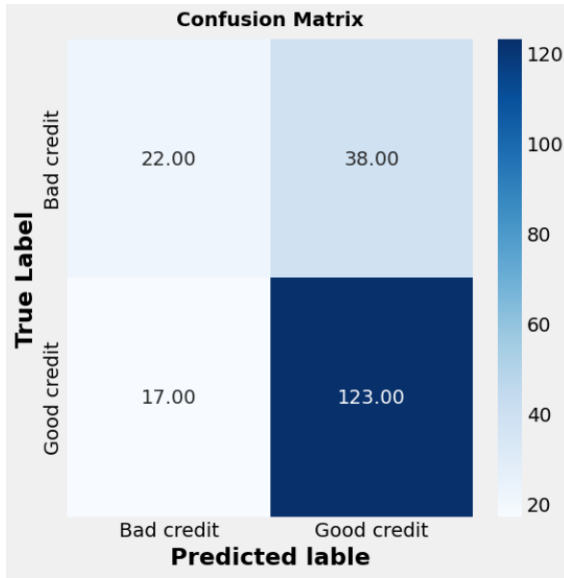


Figure 4.1: Confusion Matrix - Noise level = 0.5

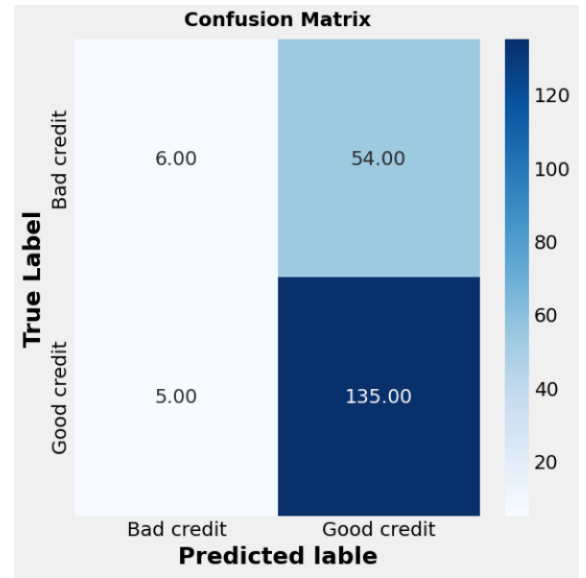


Figure 4.2: Confusion Matrix - Noise level = 19.5

Low Noise:

- With a small noise level the confusion matrix is practically the same as with no added noise.

High Noise:

- False Positives (FP): On the contrary, reaches very high level, leading to a large number of classification errors. True Negative (TN): As a consequence, TN reaches a very low level, meaning that the classifier is not able anymore to predict bad customers.

While confusion matrices allows to analyze accuracy case-by-case, another effective way to study the impact of noise over the performances is by visualizing the changes in the accuracy. This is achieved by plotting it as functions of the noise level (ϵ). From Figure 4.3, it is clear that by increasing the intensity of the noise to the dataset the overall performances of the classifier have a negative trend.

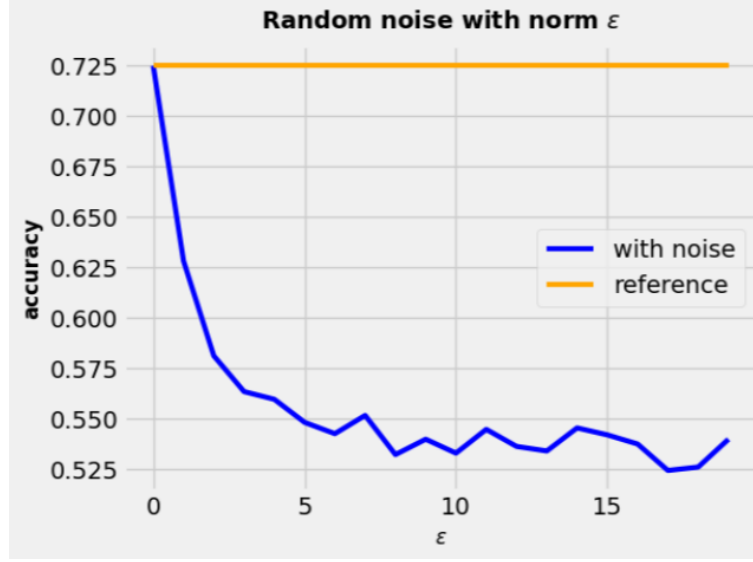


Figure 4.3: NN - Accuracy changing ϵ

4.2 Feature Specific Noise

In this section, our objective is to analyze how random noise applied to individual features affects a classifier performances. The main goal is understanding which features are the most sensible for the prediction of the labels. As in the previous case, the methodology follows a similar schema:

- In the same way as Section 4.1 we iteratively injected the same Gaussian noise to the dataset but in this case we performed the addition to the single feature.
- From 4.3 it is noticeable that starting from $\epsilon = 10$ all the accuracy values stabilize in the same interval, this means that for the "Feature Specific Noise" analysis we can limit the noise to that value without consume any additional computational resources.

After modifying a single feature (numerical or categorical) we reincorporated it into the test set. Then we re-evaluated the classifier predictions in the same way as in 4.3

4.2.1 Performance Analysis

Our expectation to this experiment are based on the concept that, since we are simulating a likely non-detectable attack, the performances of the classifier shouldn't suffer from a significant deterioration. However, it's presumable that the trend of both the metrics should follow an increasingly oscillatory trend with more instability peaks as the noise increases. For performance analysis we

will focus on evaluating the accuracy. For each modified feature we tested the classifier and plotted the oscillation of the metric in function of the compromised column. Since the number of plots is high we don't show them in the report, leaving the possibility to analyze them in the code.

To emphasize the effect of noise over the performances we graphed all the 10 iterations in the same plot to see if the instability really increases with the intensity of noise: in Figure 4.4 it is evident the difference between the two cases (we used transparency with the intermediate cases to better underline the concept).

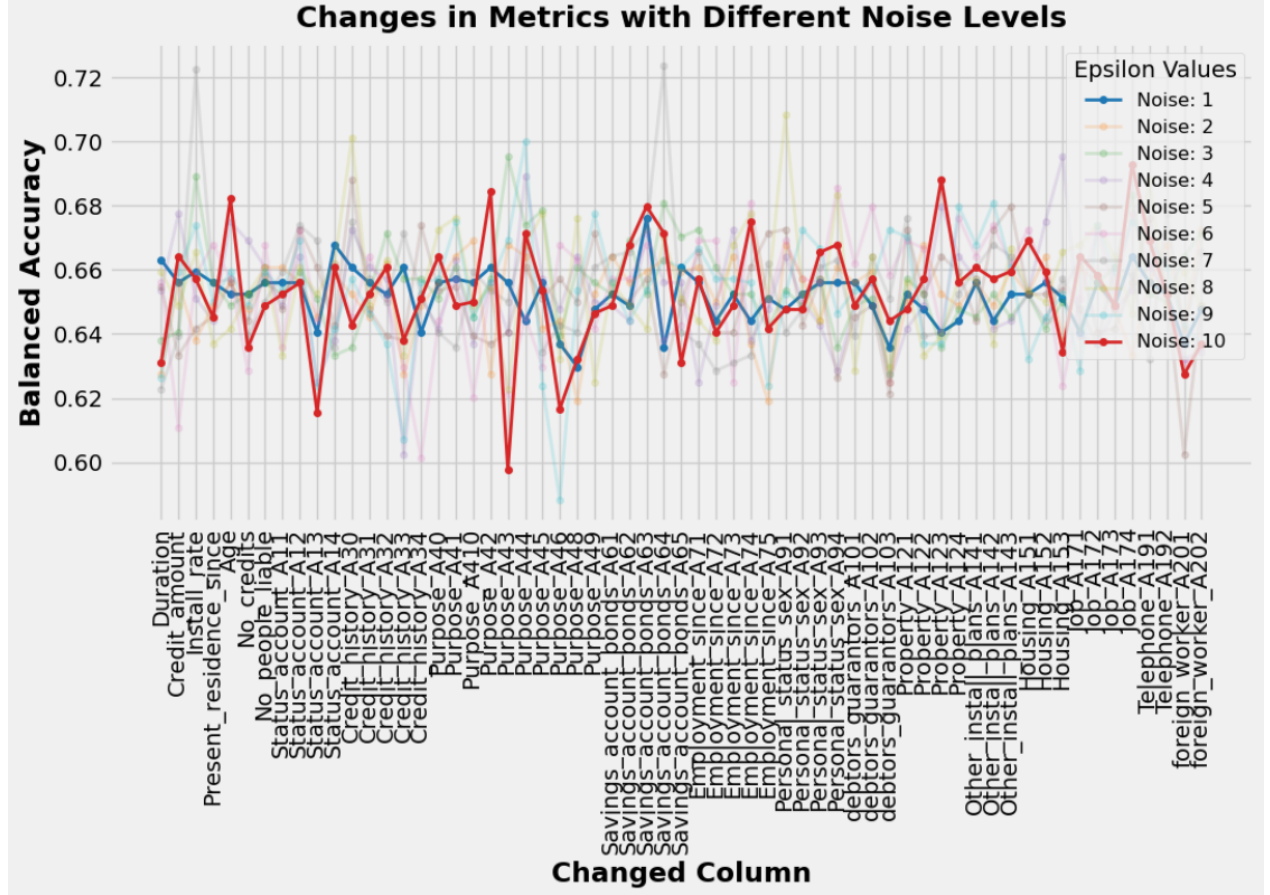


Figure 4.4: Metric trend with noise intensity variation over Features

4.2.2 Most vulnerable feature

In order to identify the most vulnerable feature, we collected all the data gathered for the plots and sifted them to find within the iterations which were the most frequent features associated to the lowest performance measures. In order to be more accurate we repeated this action for 40 cycles to see how many times the most frequent feature would be considered so. The result obtained are showed in Figure 4.5. We can conclude that "Credit Amount". This result can be considered consistent for our evaluation since the importance of this information is coherent with the main objective of the classifier.

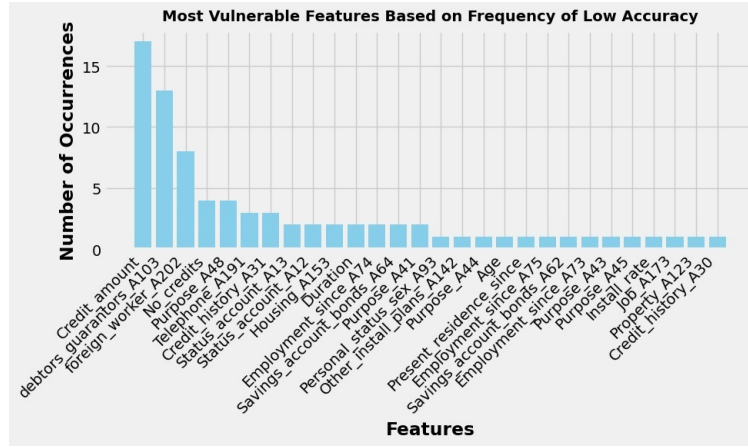


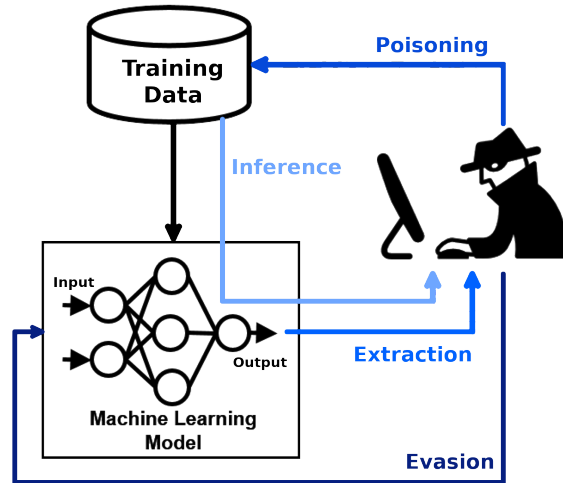
Figure 4.5: Most vulnerable feature

4.3 Adversarial Attack with ART

The **Adversarial Robustness Toolbox (ART)** is an open-source library developed by IBM for machine learning (ML) security. It is designed to help researchers and developers protect and evaluate their ML models against adversarial attacks.

We chose to implement 3 different attacks:

- Fast Gradient Sign Method (FGSM)
- Basic Iterative Method (BIM)
- Projected Gradient Descent (PGD)



ART provides tools to generate adversarial examples, implement defenses, and measure the robustness of ML models. It supports a wide range of ML frameworks, including **TensorFlow**, **PyTorch**, **Keras**, and **scikit-learn**. In particular we are using the library section dedicated to PyTorch since the model we are trying to attack was built using it.

4.3.1 Fast Gradient Sign Method (FGSM) Attack

The primary goal of FGSM is to find a small perturbation to an input dataset that will cause the neural network to misclassify the labels. The perturbation is constrained to be small to ensure

that the perturbed dataset remains similar to the original. This last point is fundamental to an Adversarial Attack which aims to poison the dataset without leaving any trace of the attack. As the name suggests it utilizes the gradient of the loss function with respect to the input data to generate the adversarial perturbation. The intuition is that the gradient indicates the direction in which the input should be modified to maximize the loss (i.e., to make the model's prediction wrong). It can use two types of norms (calculated between the original dataset and the adversarial one) to measure the size of the perturbation, indicated with ϵ

- L_2 Norm or Euclidean Distance
- L_∞ Norm or the maximum change allowed for any individual value

FGSM is one of the simplest and easiest attack to implement, it's very efficient because it can significantly decrease the model's performances. But at same time it becomes very apparent, especially for larger ϵ values. The attack was performed with 22 values of epsilon ranging from 0 to 1

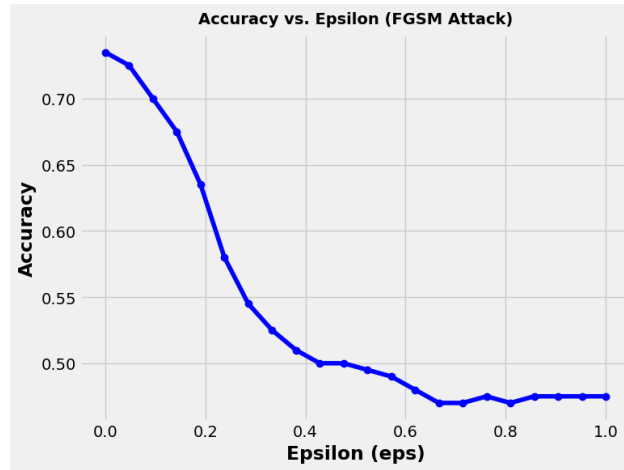


Figure 4.6: Accuracy change as epsilon increases

As we can see from 4.6 the attack is very effective in reducing the accuracy by approximately 30% after an epsilon value of 0.6.

4.3.2 Basic Iterative Method (BIM) Attack

The Basic Iterative Method (BIM) is an extension of the Fast Gradient Method for generating adversarial examples. Unlike FGSM, which applies a single perturbation step, BIM **iteratively** applies small perturbations multiple times, refining the adversarial example to be more effective.

BIM involves iteratively applying the gradient-based perturbation, rather than applying it once. This iterative process can generate stronger adversarial examples. In each iteration, a small step size (ϵ) is used to ensure that the perturbations remain small and less perceptible. After each perturbation step, the perturbed dataset is **clipped** to ensure that it remains within a valid range (e.g., 1 and 0 for a dataset normalized to have those values as max e min).

BIM is more effective than FGSM because the iterative process allows for more refined and powerful adversarial examples. It's also more flexible as it allows to change the step size (α) and the maximum perturbation (ϵ) which can be adjusted to balance between the strength of the attack and the perceptibility of the perturbation.

On the other hand the multiple iterations of BIM require multiple forward and backward passes through the model, making it way more computationally expensive than FGSM, especially increasing the value of (ϵ) which also changes the stopping criterion for the iterations. For the same reasons as the ones above it is also very much time consuming to generate adversarial examples, especially for large datasets or complex models.

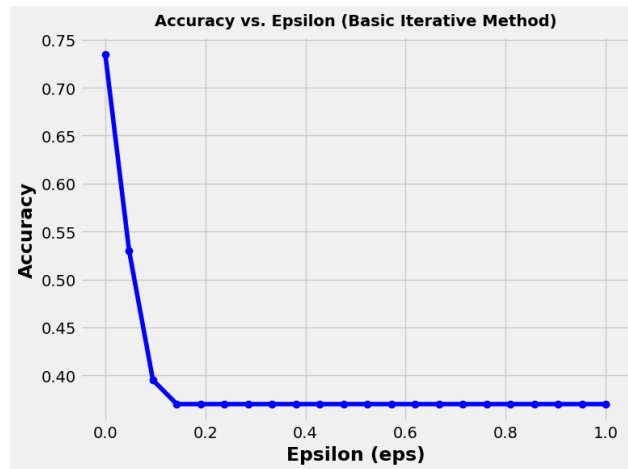


Figure 4.7: Accuracy change as epsilon increases

The greater effectiveness compared to FGSM is clear in 4.7 where the accuracy immediately drops to well below 40% only after an epsilon value of around 0.1/0.2.

4.3.3 Projected Gradient Descent (PGD) Attack

Similar to BIM, PGD applies multiple small perturbations iteratively, using the gradient of the loss function to guide the perturbations. After each iteration, PGD projects the perturbed example back onto the feasible set defined by the L_∞ or L_2 norm constraints. This ensures the perturbation does not exceed the allowed magnitude. PGD often starts from a random point within the allowed perturbation range, making it a more robust and comprehensive attack. This helps in exploring a wider adversarial space.

PGD is one of the most effective attacks for generating adversarial examples, often used as a benchmark in adversarial machine learning research. **Robustness:** The combination of iterative perturbations and projection ensures that the adversarial examples are strong and adhere to the specified norm constraints. Like BIM, PGD can be adapted for different norm constraints (L_∞ , L_2 , etc.) and can be customized with various step sizes and iteration counts.

Although PGD is very **effective** it comes with a cost, since it requires multiple iterations it makes it more computationally intensive compared to single-step attacks like FGM and of course more

time-consuming, especially for large datasets or complex models.

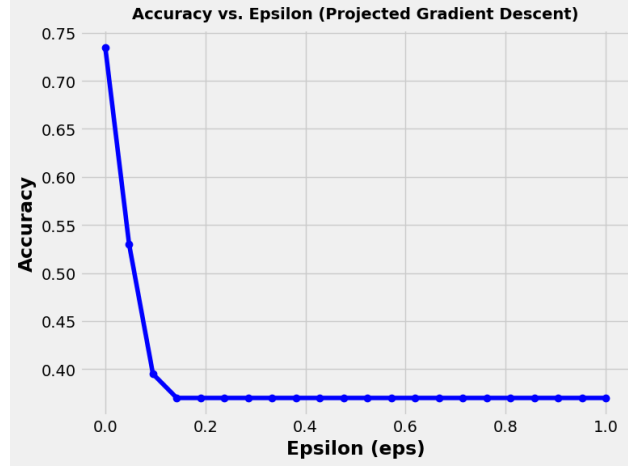


Figure 4.8: Accuracy change as epsilon increases

Here the effectiveness is similar to BIM with very similar accuracy values for similar epsilons (4.8).

4.4 Countermeasures Exploration

4.4.1 Adversarial Training

Adversarial training[4] is an effective defense mechanism aimed at enhancing the robustness of neural network models against adversarial attacks. By incorporating adversarial examples during training, the model learns to recognize and properly classify perturbed inputs, thus improving its resilience in adversarial scenarios.

To implement this technique, we first wrap the trained PyTorch model with the ART PyTorchClassifier to generate adversarial data using a simple Fast Gradient Method with an epsilon value of 0.3. Then, we combine the generated adversarial dataset with the original one and train a new model using this augmented dataset.

This approach aims to provide the model with greater robustness against adversarial attacks.

Results

Here we show the results:

Original test data accuracy: 72.00%

Adversarial test data accuracy: 60.00%

Defended test data accuracy: 88.00%

Defended adversarial test data accuracy: 82.50%

We can see that the **Adversarial Training** was useful to defend against the adversarial data and it also managed to improve the accuracy score of the original test set.

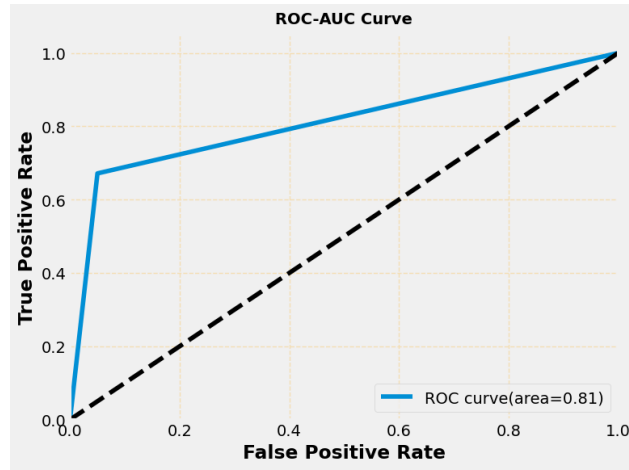


Figure 4.9: ROC-AUC Curve of the adversarial test data after the adversarial training applied

We can observe that the adversarial trained model is much more resilient against the attack with a discrete **AUC score of 0.81**.

Bibliography

- [1] *Scikit-Learn*. <https://scikit-learn.org/stable/>. [Online; accessed 01-June-2024].
- [2] *SMOTE*. <https://towardsdatascience.com/the-right-way-of-using-smote-with-cross-validation-92a8d09d00c7>. [Online; accessed 01-June-2024].
- [3] *Statlog (German Credit Data)*. <https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data>. [Online; accessed 01-June-2024].
- [4] Florian Tramèr et al. *Ensemble Adversarial Training: Attacks and Defenses*. 2020. arXiv: 1705.07204 [stat.ML]. URL: <https://arxiv.org/abs/1705.07204>.